



Instituto Tecnológico
de Buenos Aires

mPES: Esquemas de Toma de Decisión Artificial para Escenarios de Decisión Secuenciales

*Una evaluación comparativa de arquitecturas de
Aprendizaje por Refuerzo para la asignación de recursos
bajo incertidumbre en escenarios utilizando una crisis pandémica*

Alumno: Ing. Maximiliano Leonel Vega

Director: Dr. Ing. Rodrigo Ramele

Tesis de Maestría

Maestría en Ciencia de Datos

Instituto Tecnológico de Buenos Aires

2026

Resumen

Esta tesis presenta el *framework* **mPES** (*Multiple Pandemic Experiment Suite*), un entorno de trabajo multi-paquete desarrollado íntegramente en Python y diseñado como un ecosistema de experimentación para evaluar el rendimiento de agentes de Inteligencia Artificial. El objetivo central de la investigación es *evaluar empíricamente* cómo diversas arquitecturas de Aprendizaje por Refuerzo (*Reinforcement Learning*, RL) pueden optimizar la toma de decisiones en la asignación de recursos críticos durante crisis sanitarias.

Para ello, el estudio compara **siete modelos evaluados**, correspondientes a ocho paquetes implementados, que abarcan desde métodos tabulares clásicos hasta arquitecturas de vanguardia como Transformers causales y *ensembles* de redes profundas. El núcleo de la propuesta reside en superar la “miopía” de los algoritmos tradicionales mediante el modelado de trayectorias epidemiológicas como secuencias temporales, integrando además la Cuantificación de Incertidumbre (*Uncertainty Quantification*, UQ) basada en la Entropía de Shannon para dotar al sistema de transparencia y confiabilidad.

En última instancia, esta investigación busca *generalizar* el problema de la distribución de recursos limitados ante dinámicas de propagación complejas y no lineales. Los resultados empíricos sobre un *benchmark* de 22 escenarios evaluados, compuesto por una condición *in-distribution* y 21 condiciones *out-of-distribution* ($n = 64$ secuencias por escenario), confirman *parcialmente* la hipótesis de trabajo: el Transformer causal (**pes_trf**, $\bar{r} = 0,927$) es la mejor arquitectura *individual*, pero el *ensemble* ponderado (**pes_ens**, $\bar{r} = 0,937$, $\sigma = 0,035$) lo supera en el rendimiento agregado y la consistencia entre secuencias. Los hiperparámetros de los modelos optimizables fueron sintonizados mediante Optimización Bayesiana con el estimador estructurado en árbol de Parzen (TPE); **pes_base** conserva parámetros fijos y **pes_ens** no requiere entrenamiento adicional.

Palabras clave: mPES, Aprendizaje por Refuerzo, Inteligencia Artificial, Transformer Causal, *Deep Ensembles*, Cuantificación de Incertidumbre, Entropía de Shannon, Optimización Bayesiana, Asignación de Recursos, Secuencias Temporales, Evaluación *Out-of-Distribution*.

Índice

1. Introducción	1
1.1. Motivación	1
1.2. Definición del Problema	2
1.3. Hipótesis de Trabajo	2
1.4. Contribuciones	2
1.5. Estructura del Documento	3
2. Marco Teórico	3
2.1. Procesos de Decisión de Markov	4
2.2. Q-Learning tabular	4
2.2.1. Double Q-Learning	4
2.2.2. Reward Shaping Basado en Potencial (PBRs)	5
2.3. Deep Q-Networks (DQN)	5
2.4. Redes recurrentes y LSTM	5
2.5. Políticas, Gradientes y Actor-Critic	6
2.6. Atención y el Transformer	6
2.7. Cuantificación de Incertidumbre y Entropía de Shannon	7
2.8. Optimización Bayesiana de Hiperparámetros	7
2.9. Distribución de entrenamiento y generalización	8
2.10. Comparación Estadística de Agentes	8
3. Estado de la Cuestión	9
3.1. De la Metacognición Humana a la IA	9
3.2. Aprendizaje por Refuerzo en la Gestión de Crisis	10
3.3. El Cambio de Paradigma: Transformers y Modelado de Secuencias	10
3.4. IA Explicable (XAI) y Cuantificación de Incertidumbre	11
3.5. Generalización <i>out-of-distribution</i> y <i>benchmarks</i> de robustez	11
3.6. Posición de este trabajo	12
4. Materiales y Métodos	12
4.1. Repositorio y reproducibilidad	13

4.2. El entorno: <i>Pandemic Scenario</i>	13
4.2.1. Definición formal	13
4.2.2. Dinámica y recompensa	14
4.2.3. Estructura del benchmark	14
4.3. Los ocho paquetes algorítmicos	14
4.3.1. Familia tabular	15
4.3.1.1. <code>pes_base.</code>	15
4.3.1.2. <code>pes_q1.</code>	16
4.3.1.3. <code>pes_dq1.</code>	16
4.3.2. Familia profunda	17
4.3.2.1. <code>pes_dqn.</code>	17
4.3.2.2. <code>pes_rdqn.</code>	17
4.3.2.3. <code>pes_a2c.</code>	17
4.3.2.4. <code>pes_trf.</code>	17
4.3.2.5. <code>pes_ens.</code>	18
4.4. Pipeline de optimización bayesiana	18
4.5. Calibración con un agente aleatorio	19
4.6. Trazas internas del agente Transformer	20
4.7. Catálogo de escenarios OOD	20
4.8. Protocolo de entrenamiento y evaluación	23
5. Resultados	23
5.1. Familias de variaciones evaluadas	24
5.2. Visión global	24
5.3. Reducción de la severidad normalizada	25
5.4. Comportamiento por familia algorítmica	27
5.5. Análisis de Cuantificación de Incertidumbre	28
5.6. Salidas automáticas del orquestador	28
5.7. Curvas de entrenamiento por algoritmo	29
5.8. Curvas por escenario	33
5.9. Comportamiento bajo los estresores universales	33
6. Discusión	35

6.1. Factores compatibles con el mejor desempeño del Transformer	35
6.2. Entropía de Shannon como proxy de confianza algorítmica	35
6.3. El ensemble como mejor sistema del benchmark	36
6.4. Los tres estresores universales	36
6.5. El falso aprobado de <code>pes_a2c</code>	37
7. Conclusiones	37
7.1. Mensajes principales	37
7.2. Limitaciones del estudio	38
7.3. Trabajo futuro	39
7.4. Cierre	39
8. Agradecimientos	40
Referencias Bibliográficas	41
Apéndice	44
A. Comandos de reproducción	44
B. Orquestador del benchmark	44
C. Estresores universales	45

Índice de figuras

4.1. Severidad cruda del decisor aleatorio	20
4.2. Trazas internas del Transformer	21
5.1. Resultados del modelo base	29
5.2. Resultados de Q-Learning optimizado	30
5.3. Resultados de Double Q-Learning	30
5.4. Resultados de DQN	31
5.5. Resultados de RDQN	31
5.6. Resultados de A2C	32
5.7. Resultados del Transformer	32
5.8. Resultados del ensemble	33
5.9. Curvas adicionales por familia de perturbación	34
5.10. Curvas bajo estresores universales	34

Índice de cuadros

4.1. Arquitecturas evaluadas	15
4.2. Catálogo de escenarios OOD	22
5.1. Desempeño normalizado medio por modelo	25
5.2. Desempeño por bloque de evaluación	26
5.3. Comparaciones entre modelos	26
A.1. Comandos de reproducción	44

1. Introducción

La toma de decisiones complejas y secuenciales bajo incertidumbre constituye un desafío central para los sistemas de inteligencia artificial. En estos problemas, cada acción modifica el estado del entorno y condiciona las decisiones posteriores, por lo que una política eficaz debe anticipar consecuencias acumulativas y asignar recursos limitados a lo largo del tiempo. El *framework* **mPES** surge como una plataforma para comparar sistemáticamente arquitecturas de Aprendizaje por Refuerzo en este tipo de escenarios de decisión. La crisis pandémica se utiliza como un *testbench*: un caso concreto, exigente y reproducible que permite estudiar la adaptabilidad de los agentes frente a dinámicas no lineales, información incierta y cambios fuera de la distribución de entrenamiento. Para ello, mPES extiende el entorno original *Pandemic Experiment Scenario* (PES) del BCI-NE Lab de la Universidad de Essex [1].

1.1. Motivación

La crisis sanitaria provocada por el COVID-19 ofrece un caso representativo de este tipo de decisiones. Gobiernos, hospitales y cadenas logísticas deben ponderar beneficios inmediatos contra costos a largo plazo, mientras que cada asignación altera la evolución posterior del sistema. En este trabajo, la pandemia no se considera el límite del problema, sino el escenario experimental que permite observar y medir esas propiedades. La investigación busca así evaluar qué modelos matemáticos y computacionales producen decisiones más adaptativas en escenarios complejos y secuenciales, utilizando la distribución de recursos durante una crisis pandémica como banco de pruebas.

El enfoque metodológico de esta tesis está deliberadamente impregnado de la disciplina del *desarrollo de producto* propia de la ingeniería: trazabilidad de cada artefacto, reproducibilidad estricta, control de versiones del entorno computacional y validación *out-of-distribution* como requisito de aceptación. Bajo esta óptica, mPES no se concibe como un experimento aislado sino como un *sistema* — ocho paquetes implementados, siete modelos evaluados, un esquema de evaluación común y un orquestador reproducible — pensado para estudiar arquitecturas de decisión secuencial en dominios de alta complejidad.

1.2. Definición del Problema

El entorno computacional desarrollado implementa, como escenario de prueba, una crisis pandémica donde un tomador de decisiones (un agente de RL) debe distribuir un *presupuesto limitado* de recursos médicos o logísticos entre diversas ciudades afectadas simultáneamente. Tres rasgos cualitativos definen el escenario: la *dinámica acumulativa* (la severidad no atendida crece de forma exponencial en los pasos subsiguientes), la *intervención temprana* (los recursos asignados generan un impacto sostenido que premia a los modelos capaces de anticipar) y una *evolución matemática* regida por la ley exponencial-corregida

$$S'_i = \max(0, \beta S_i - \alpha a_i) \quad \alpha = 0,4 \quad \beta = 1,4$$

La formulación completa como MDP finito y el *benchmark* fijo de 64 secuencias compartido por todos los enfoques se detallan en la Sec. 4.

1.3. Hipótesis de Trabajo

El trabajo se organiza alrededor de una hipótesis falsable desdoblada en dos niveles:

H1a (arquitectónica). *Entre las arquitecturas individuales evaluadas, el modelo Transformer causal alcanza el mayor desempeño medio en la toma de decisiones secuenciales bajo incertidumbre, utilizando la asignación de recursos durante una pandemia como escenario de prueba, especialmente en condiciones fuera de distribución.*

H1b (sistémica). *El Transformer, considerado como modelo individual, constituye el mejor modelo del benchmark según rendimiento y robustez, por encima de los modelos individuales de referencia.*

1.4. Contribuciones

Las principales contribuciones de esta tesis son:

1. Un *benchmark* de RL reutilizable — *workspace* **mPES** — donde **ocho variantes algorítmicas** comparten el mismo entorno Gymnasium, el mismo *pipeline* de optimización bayesiana con Optuna y el mismo esquema de evaluación sobre $n = 64$

secuencias *fijas*.

2. Una comparación cuantitativa y estadísticamente fundamentada entre métodos tabulares (`pes_base`, `pes_q1`, `pes_dql`) y arquitecturas profundas (`pes_dqn`, `pes_rdqn`, `pes_a2c`, `pes_trf`, `pes_ens`), abarcando los principales paradigmas del RL contemporáneo: valor tabular, valor profundo, recurrente, gradiente de política, atención causal y promediado de modelos.
3. Una matriz de evaluación de $7 \times 22 = 154$ celdas, con una condición *in-distribution* y 21 condiciones *out-of-distribution*, que estresa cada agente contra perturbaciones sistemáticas de severidad, longitud, conjuntas y estructurales, junto con un protocolo estadístico unificado (d de Cohen, t de Welch y divergencia KL entre políticas).
4. La integración de la **Entropía de Shannon** como métrica de *Cuantificación de Incertidumbre* (UQ), estableciendo un puente algorítmico con las teorías neurocognitivas de la confianza humana.
5. Una implementación reproducible y multi-plataforma (Windows 10 y Ubuntu, Python 3.12, TensorFlow 2.21) liberada junto con la tesis.

1.5. Estructura del Documento

El resto del documento se organiza como sigue. La Sección 2 introduce el marco teórico (MDPs, Q-Learning, redes recurrentes, atención, ensembles, entropía de Shannon). La Sección 3 presenta el estado de la cuestión (legado neurocientífico de Oxford, RL en gestión de crisis, Transformers en RL, XAI y UQ). La Sección 4 describe la metodología: el escenario pandémico, las ocho arquitecturas implementadas y el *pipeline* experimental. La Sección 5 reporta los resultados empíricos. La Sección 6 interpreta esos resultados a la luz de la hipótesis y la Sección 7 cierra el trabajo.

2. Marco Teórico

Este capítulo introduce la maquinaria formal utilizada a lo largo de la tesis: Procesos de Decisión de Markov (MDPs), *Reinforcement Learning* basado en valor y en gradiente de política, aproximación profunda de funciones, codificadores recurrentes y basados en

atención, optimización bayesiana de hiperparámetros y, finalmente, la Entropía de Shannon como métrica de Cuantificación de Incertidumbre (UQ). El tratamiento es deliberadamente compacto; las referencias canónicas son Sutton & Barto [2].

2.1. Procesos de Decisión de Markov

Un MDP finito es una tupla $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$ donde $\mathcal{S}$ es un espacio de estados finito, $\mathcal{A}$ un espacio de acciones finito, $P(s' | s, a)$ un núcleo de transición, $R(s, a, s')$ una función de recompensa acotada y $\gamma \in [0, 1)$ el factor de descuento. Bajo una política $\pi(a | s)$ el agente genera una trayectoria $\tau = (s_0, a_0, r_1, s_1, a_1, \dots)$ cuyo *retorno descontado* desde el tiempo t es

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \quad (2.1)$$

La función de valor de estado–acción de la política es $Q^\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a]$ y la política óptima satisface la ecuación de Bellman,

$$Q^*(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right] \quad (2.2)$$

2.2. Q-Learning tabular

Cuando $\mathcal{S}$ y $\mathcal{A}$ son enumerables, Q se almacena como una tabla y se aplica la actualización clásica de Watkins [3]:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad (2.3)$$

con tasa de aprendizaje α y exploración ε -greedy.

2.2.1. Double Q-Learning

El operador $\max$ en la Ecuación (2.3) introduce un sesgo positivo en la estimación del valor. *Double Q-Learning* [4] mantiene dos tablas Q^A y Q^B actualizadas alternativamente, desacoplando la selección y la evaluación de la acción:

$$Q^A(s, a) \leftarrow Q^A(s, a) + \alpha [r + \gamma Q^B(s', \arg \max_{a'} Q^A(s', a')) - Q^A(s, a)] \quad (2.4)$$

2.2.2. Reward Shaping Basado en Potencial (PBRS)

PBRS [5] añade un término $F(s, s') = \gamma\Phi(s') - \Phi(s)$ a la recompensa sin alterar el conjunto de políticas óptimas. Es la única familia de *shaping* con garantía teórica de invarianza y se utiliza en `pes_dql` para acelerar la convergencia frente a una señal de costo final dispersa.

2.3. Deep Q-Networks (DQN)

Cuando el espacio de estados es grande, $Q(s, a; \theta)$ se aproxima con una red neuronal. El *Deep Q-Network* [6] estabiliza el entrenamiento con dos ingredientes: (i) un *buffer de repetición de experiencia* [7] del cual se muestrean minilotes uniformemente, y (ii) una *red objetivo* $Q(\cdot; \theta^-)$ cuyos parámetros se copian desde θ cada C pasos. La función de costo es

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right] \quad (2.5)$$

La red objetivo desacopla el blanco de la regresión y se actualiza por copia dura cada C pasos o, alternativamente, por *soft update* $\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^-$ con $\tau \ll 1$. La pérdida se optimiza con Adam [8] y, opcionalmente, con un *schedule* tipo *warm restarts*. Para regularizar la red se utiliza *dropout*.

2.4. Redes recurrentes y LSTM

Cuando el estado observado no es markoviano — por ejemplo, cuando la dinámica de severidad acumula información histórica no representada en s_t — es habitual recurrir a codificadores recurrentes. La *Long Short-Term Memory* (LSTM) [9] introduce compuertas i_t, f_t, o_t que controlan cómo el estado de celda c_t acumula información a largo plazo:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad h_t = o_t \odot \tanh(c_t) \quad (2.6)$$

Hausknecht y Stone [10] demostraron que incorporar una capa LSTM dentro de un DQN (*Deep Recurrent Q-Network*, DRQN) mejora el desempeño en entornos parcialmente observables. El paquete `pes_rdn` adopta exactamente esta estrategia.

2.5. Políticas, Gradientes y Actor–Critic

Los métodos de *gradiente de política* [11] parametrizan la política $\pi_\theta(a \mid s)$ y actualizan θ a lo largo del gradiente del retorno esperado:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a \mid s) A^{\pi_\theta}(s, a)] \quad (2.7)$$

con la función de *ventaja* $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$. El *Advantage Actor–Critic* (A2C) [12] entrena simultáneamente un actor que produce π_θ y un crítico que estima V_ϕ . La estimación de la ventaja puede refinarse con *Generalized Advantage Estimation* (GAE),

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{k=0}^{\infty} (\gamma \lambda)^k \delta_{t+k}, \quad \delta_t = r_{t+1} + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) \quad (2.8)$$

que interpola entre Monte Carlo ($\lambda=1$, bajo sesgo / alta varianza) y *bootstrapping* de un paso ($\lambda=0$). El actor se regulariza con un término de entropía $-\beta H(\pi_\theta(\cdot \mid s))$ que combate el colapso prematuro de la política. Variantes más estables como PPO introducen un objetivo recortado para evitar pasos demasiado agresivos; el estudio empírico de Andrychowicz *et al.* [13] resume las decisiones de diseño que más inciden en el resultado.

2.6. Atención y el Transformer

El *Transformer* [14] reemplaza la recurrencia por *atención de producto escalar escalado*. Dadas consultas Q , claves K y valores V ,

$$\text{Atención}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (2.9)$$

En un entorno autorregresivo, una máscara triangular $M_{ij} = -\infty$ para $j > i$ prohíbe que

una posición atienda al futuro:

$$\text{Atención}_{\text{causal}}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V \quad (2.10)$$

En RL, trabajos recientes [15, 16] demuestran que los codificadores Transformer pueden reemplazar funciones de valor recurrentes con mejor capacidad de captura de dependencias de largo plazo, ofreciendo una solución al problema de la “miopía temporal” que afecta a los métodos tabulares en escenarios de inercia epidemiológica. En esta tesis cada bloque Transformer incluye conexiones residuales y *layer normalization*, indispensable para estabilizar el entrenamiento de políticas profundas.

2.7. Cuantificación de Incertidumbre y Entropía de Shannon

Dada una distribución de probabilidad discreta $p(a \mid s)$ sobre $|\mathcal{A}|$ acciones, la *entropía de Shannon* es

$$H(p) = - \sum_{a \in \mathcal{A}} p(a \mid s) \log p(a \mid s) \quad (2.11)$$

Normalizada por $\log |\mathcal{A}|$, $H \in [0, 1]$ se interpreta como una medida de *incertidumbre intrínseca* del agente: $H \rightarrow 0$ significa que el modelo concentra toda su masa en una sola acción (alta confianza); $H \rightarrow 1$ indica una distribución uniforme (máxima duda). En esta tesis se propone usar $1 - H_{\text{norm}}$ como proxy de *confianza metacognitiva* del agente, en analogía con los marcadores neuronales de confianza identificados por Boldt & Yeung [17] y revisados por Fleming [18].

2.8. Optimización Bayesiana de Hiperparámetros

Un agente de RL tiene varios hiperparámetros no diferenciables (α , γ , tamaño del *replay buffer*, *schedule* de ε , anchura de red, etc.) cuyas interacciones son altamente no lineales. La *Optimización Bayesiana* trata el puntaje de validación como una función de caja negra ruidosa y la explora con un *surrogate* probabilístico. Se utiliza el estimador estructurado

en árbol de Parzen (TPE) [19] implementado en Optuna [20], que modela $p(x \mid y < y^*)$ y $p(x \mid y \geq y^*)$ con KDEs separados. Una *Kernel Density Estimator* (KDE) es un estimador no paramétrico de densidad que representa la distribución de una muestra mediante la suma de funciones núcleo centradas en cada observación. Para una muestra unidimensional $x_1, \dots, x_n$, la estimación es

$$\hat{p}_h(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right) \quad (2.12)$$

donde K es la función núcleo y $h > 0$ el ancho de banda, que controla el grado de suavizado. En TPE, estas estimaciones se construyen por separado para las configuraciones con rendimiento inferior a y^* y para el resto; la razón entre ambas densidades orienta la selección de nuevos hiperparámetros.

2.9. Distribución de entrenamiento y generalización

Sea $P_{\text{train}}(s, a, r, s')$ la distribución de transiciones observadas durante el entrenamiento y sea $P_{\text{test}}(s, a, r, s')$ la distribución utilizada para evaluar la política aprendida. Una evaluación *in-distribution* (ID) conserva las mismas leyes de generación que el entrenamiento, de modo que

$$P_{\text{test}} = P_{\text{train}} \quad (2.13)$$

Una evaluación *out-of-distribution* (OOD), en cambio, modifica una o más propiedades de esa distribución, por lo que $P_{\text{test}} \neq P_{\text{train}}$. El cambio puede afectar la distribución de estados o entradas (*covariate shift*), la relación entre entradas y recompensas, o la dinámica de transición. En consecuencia, el rendimiento OOD mide la capacidad de generalización y robustez de la política, y no solamente su ajuste a las condiciones conocidas durante el entrenamiento.

2.10. Comparación Estadística de Agentes

Para comparar dos agentes estocásticos A y B sobre n semillas independientes se reportan tres cantidades:

Tamaño del efecto: d de Cohen [21] clasificado como pequeño ($|d| \geq 0,2$), medio ($|d| \geq 0,5$) y grande ($|d| \geq 0,8$).

$$d = \frac{\bar{r}_A - \bar{r}_B}{\sqrt{(\sigma_A^2 + \sigma_B^2)/2}} \quad (2.14)$$

Significancia: t de Welch [22] con grados de libertad de Welch–Satterthwaite.

Cambio de política: Divergencia de Kullback–Leibler entre los histogramas de acción de ambos agentes, $D_{\text{KL}}(p_A \parallel p_B)$.

3. Estado de la Cuestión

La presente investigación se sitúa en la intersección de la epidemiología computacional [23], la neurociencia cognitiva de la confianza y el aprendizaje profundo secuencial. El estado del arte actual revela un creciente interés en sistemas de soporte de decisiones que no sólo optimicen una métrica de rendimiento, sino que incorporen nociones de *confianza*.

3.1. De la Metacognición Humana a la IA

El punto de partida conceptual de este trabajo es un *dataset* público (ds004477[24]) originado en una colaboración británica entre varias universidades e implementado y liberado por el BCI-NE Lab de la Universidad de Essex como el entorno *Pandemic Experiment Scenario* (PES) [1]. Esta línea de investigación examinó las firmas neuronales de la confianza humana ante decisiones bajo presión. Estudios fundamentales como los de Boldt & Yeung [17] identificaron que el cerebro humano utiliza biomarcadores específicos (observables mediante EEG) para monitorear errores y ajustar la certeza de una acción. Revisiones recientes [18] formalizan la relación entre confianza, análisis secuencial y teoría de detección de señales, mostrando que estos biomarcadores son cruciales en tareas de alta carga cognitiva.

La perspectiva de los equipos humano-máquina amplía este puente al considerar que la toma de decisiones puede distribuirse entre personas y sistemas artificiales, de modo que la confianza y la adaptación constituyen propiedades relevantes del equipo completo [25].

La originalidad de mPES radica en *tomar esta premisa neurofisiológica y trasladarla al plano algorítmico*: sustituir la confianza extraída del EEG por la Entropía de Shannon

(Ecuación 2.11) como métrica de incertidumbre intrínseca del agente de IA.

3.2. Aprendizaje por Refuerzo en la Gestión de Crisis

La literatura en RL ha evolucionado desde métodos tabulares como el Q-Learning [3] hacia aproximaciones capaces de manejar la alta dimensionalidad de los problemas del mundo real [2].

- **Deep Q-Networks (DQN).** La introducción del *Experience Replay* y las *Target Networks* por Mnih *et al.* permitió estabilizar el aprendizaje en entornos complejos. Sin embargo, en el contexto epidemiológico, estos modelos suelen sufrir de un *sesgo de sobreestimación* que el Double Q-Learning intenta mitigar mediante el desacoplamiento de la selección y evaluación de acciones.
- **Mejoras de optimización.** El uso de Adam con decaimiento cíclico y *schedules* adaptativos de exploración como RBED (*Reward Based Epsilon Decay*) aceleran la convergencia.
- **Actor–Critic (A2C).** La arquitectura Actor–Crítico se ha identificado como una solución robusta para reducir la varianza en los gradientes de política. Su capacidad para modelar una función de ventaja es fundamental para gestionar la volatilidad extrema de los datos de contagio.

3.3. El Cambio de Paradigma: Transformers y Modelado de Secuencias

El mayor obstáculo en la gestión de pandemias es la *inercia temporal*: una decisión hoy afecta la curva de severidad acumulada semanas después. Los modelos tradicionales basados en MDPs suelen ser “miopes”, ya que priorizan el estado inmediato.

La propuesta de Vaswani *et al.* [14] con la arquitectura Transformer y su posterior adaptación al RL mediante el *Decision Transformer* [16], representa un cambio de paradigma. Al tratar el RL como un problema de predicción de secuencias y utilizar mecanismos de *auto-atención* (Ecuación 2.10), el agente puede capturar dependencias de largo plazo, entendiendo la *trayectoria completa* de la pandemia en lugar de observar

fotografías aisladas del presente. Trabajos como el *Gated Transformer-XL* [15] mostraron que, con compuertas e inicialización adecuadas, los Transformers igualan o superan a las redes LSTM en tareas que requieren memoria.

3.4. IA Explicable (XAI) y Cuantificación de Incertidumbre

La adopción de IA en salud pública está limitada por la opacidad de los modelos. La Cuantificación de Incertidumbre (UQ) mediante el uso de entropía en Transformers permite que el sistema informe cuándo “no está seguro” de una decisión.

Si el agente detecta una alta entropía (incertidumbre), emula el proceso de duda humana, permitiendo una intervención supervisada más segura en escenarios de crisis. La originalidad de esta tesis consiste, precisamente, en *operacionalizar* ese puente neurocognitivo mediante la Entropía de Shannon sobre la distribución de salida del Transformer causal.

3.5. Generalización *out-of-distribution* y *benchmarks* de robustez

Una línea creciente de trabajo en RL profundo ha denunciado que las métricas reportadas en la distribución de entrenamiento sobreestiman sistemáticamente la calidad de las políticas aprendidas. Henderson *et al.* [26] mostraron que los algoritmos más populares de RL profundo son extremadamente sensibles a la semilla aleatoria, los hiperparámetros y los detalles de implementación; Cobbe *et al.* [27] demostraron, sobre la suite ProcGen, que agentes que parecían aprender una tarea en realidad memorizaban niveles concretos y fracasaban al evaluarse fuera de la distribución; Packer *et al.* [28] formalizaron este sesgo como una taxonomía de *shifts* (parámetros extrapolados, escenarios estructurales y perturbaciones conjuntas) y Kirk *et al.* [29] sintetizaron el estado del arte de la generalización en RL en su revisión *Survey of Generalisation in Deep Reinforcement Learning*, recomendando explícitamente reportar matrices algoritmo $\times$ escenario con cuantificación de significancia estadística.

El diseño del benchmark agregado de mPES (Sec. 4) se apoya directamente en estas recomendaciones: $7 \times 22 = 154$ celdas, cuatro familias de perturbación, contrastes de Welch reportados como $-\log_{10} p$, columnas estructurales como control negativo y degradación Δr frente a la condición empírica. Esta elección metodológica es uno de los aportes diferenciales del trabajo: en lugar de declarar un ganador en la condición empírica y dar por concluida la evaluación, el sweep completo revela qué arquitecturas degradan suavemente y cuáles sufren colapsos catastróficos (Sec. 6.5).

3.6. Posición de este trabajo

Comparado con la literatura precedente, la contribución de esta tesis es:

- **Ocho arquitecturas, un mismo esquema experimental.** Una comparación *lado a lado* que abarca métodos tabulares (Q-Learning, Double Q-Learning), profundos densos (DQN), recurrentes (Recurrent DQN con LSTM), actor-crítico (A2C), Transformer causal y *deep ensembles*, todos entrenados sobre el mismo MDP y evaluados con $n = 64$ secuencias por escenario a través de 22 condiciones fuera de distribución.
- **UQ algorítmica.** La adopción explícita de la Entropía de Shannon como proxy de confianza, en línea con el estudio algorítmico de la confianza humana.
- **Rigor estadístico.** Cada diferencia reportada viene acompañada de un tamaño de efecto (d de Cohen), una significancia (p de Welch) y una divergencia de Kullback–Leibler sobre la distribución de acciones que detecta colapsos silenciosos de política (Sec. 6.5).

4. Materiales y Métodos

Este capítulo describe el aparato experimental: el *workspace* mPES, la definición formal del entorno, los ocho paquetes algorítmicos comparados, el pipeline de optimización bayesiana de hiperparámetros y el protocolo de evaluación.

4.1. Repositorio y reproducibilidad

Todos los experimentos se ejecutan dentro del *workspace* mPES [30], derivado y extendido a partir del repositorio base PES del grupo BCI-NE [1], del cual hereda la definición original del entorno *Pandemic Scenario* y la infraestructura tabular de Q-Learning. Es un proyecto Python organizado en dos familias: `tabular/` (métodos enumerativos) y `ml/` (aprendizaje profundo). Cada paquete es autocontenido: provee su propio entorno `gymnasium` [31], su rutina de entrenamiento, su módulo de optimización bayesiana y su carpeta `outputs/` versionada por fecha. Las dependencias principales son TensorFlow 2.21, Keras 3.13, NumPy 2.4, Optuna 4.7, Gymnasium 1.2 y SciPy 1.17.

4.2. El entorno: *Pandemic Scenario*

4.2.1. Definición formal

El entorno modela una pandemia simplificada como un MDP finito en el que el agente decide, en cada *trial*, cuántas unidades de un recurso escaso destinar a contener un brote. El espacio de estados es

La formulación como un sistema dinámico de decisiones se relaciona con los modelos formales en los que las decisiones actuales modifican el estado y las opciones disponibles en etapas posteriores [32]. En este trabajo, esa estructura se instancia en un modelo epidemiológico simplificado utilizado como banco de pruebas, no como una reproducción de una pandemia real.

$$\mathcal{S} = \{(R, t, S) : R \in \{0, \dots, 30\} \ t \in \{0, \dots, T\} \ S \in \{0, \dots, S_{\text{máx}}\}\} \quad (4.1)$$

con R los recursos restantes, t el índice de *trial* (con $t = 0$ en el estado inicial) y S la severidad actual; $|\mathcal{S}| = 3\,410$ estados alcanzables. El presupuesto total de cada secuencia es de 39 recursos; como 9 se asignan previamente, el estado observa $R \in \{0, \dots, 30\}$ recursos disponibles. El espacio de acciones es $\mathcal{A} = \{0, 1, \dots, 10\}$ (asignaciones que exceden R son enmascaradas).

4.2.2. Dinámica y recompensa

La dinámica de la severidad sigue la ley exponencial-corregida

$$S_{t+1} = \max(0, \beta \cdot S_t - \alpha \cdot a_t) \quad \alpha = 0,4 \quad \beta = 1,4 \quad (4.2)$$

con $\beta > 1$ capturando el crecimiento natural del contagio (inercia) y α la eficacia marginal de cada unidad asignada. La recompensa instantánea es

$$r_t = -S_{t+1} \quad (4.3)$$

de modo que la política óptima minimiza la severidad acumulada total del escenario simulado.

4.2.3. Estructura del benchmark

Cada experimento del *workspace* mPES se organiza jerárquicamente en **8 bloques** de **8 secuencias** de **3–10 trials** cada una, totalizando ≈ 360 trials por ejecución. La evaluación final se realiza sobre $n = 64$ secuencias fijas (8×8 , semilla = 42), garantizando que los siete modelos evaluados enfrenten el *mismo* conjunto de escenarios. El catálogo incluye **22 escenarios**: una condición de referencia *in-distribution* y 21 condiciones *out-of-distribution*, agrupadas en cuatro familias — *severidad* (9), *longitud* (5), *conjunta* (4) y *estructural* (3) — de modo que el barrido completo del orquestador (`general/scripts/orchestrate.py`) abarca $7 \times 22 = 154$ celdas (todos los paquetes excepto `pes_base`, que actúa como cota inferior de referencia y no requiere optimización). El decisor trivial se utiliza como referencia para interpretar posteriormente el desempeño de las arquitecturas optimizadas.

4.3. Los ocho paquetes algorítmicos

La Tabla 4.1 resume las ocho arquitecturas evaluadas. El repositorio mPES organiza estas variantes en dos familias: *tabular* (métodos enumerativos con representación exacta del MDP) y *profunda* (redes neuronales entrenadas por descenso de gradiente). Cada

paquete comparte el entorno Gymnasium descrito en la Ecuación (4.2), el bucle externo de optimización bayesiana con Optuna y el mismo conjunto fijo de 64 secuencias de validación, garantizando una comparación rigurosa *ceteris paribus*.

Cuadro 4.1: Paquetes y familias algorítmicas incluidos en mPES, con la característica distintiva de cada implementación. El benchmark evalúa siete modelos; `pes_base` se conserva como referencia y `pes_ens` combina tres modelos profundos.

Paquete	Familia	Característica distintiva
<code>pes_base</code>	Tabular	Q-Learning de referencia.
<code>pes_q1</code>	Tabular	Q-Learning + Optuna/TPE.
<code>pes_dq1</code>	Tabular	Double Q-Learning + PBRS + ε <i>warm-up</i> .
<code>pes_dqn</code>	Profundo	DQN con <i>replay</i> y red objetivo.
<code>pes_rdqn</code>	Profundo	Recurrent DQN con codificador LSTM.
<code>pes_a2c</code>	Profundo	Actor-Crítico (A2C).
<code>pes_trf</code>	Profundo	Codificador Transformer causal sobre historia.
<code>pes_ens</code>	Profundo	<i>Soft voting</i> de DQN+RDQN+TRF.

4.3.1. Familia tabular

Los tres agentes tabulares representan explícitamente la función $Q(s, a)$ mediante una tabla. El estado es $s = (R, t, S) \in \{0, \dots, 30\} \times \{0, \dots, 10\} \times \{0, \dots, 9\}$, por lo que la tabla tiene $31 \times 11 \times 10 \times 11 = 37\,510$ valores. Las acciones son $a \in \{0, \dots, 10\}$ y las que exceden los recursos disponibles se enmascaran. En cada paso se selecciona una acción mediante una política ε -greedy, se observa (s', r) y se actualiza la entrada visitada. Para Q-Learning estándar, la actualización es

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (4.4)$$

salvo en un estado terminal, donde se utiliza $Q(s, a) \leftarrow r$. Las tablas se inicializan con valores de una distribución uniforme $\mathcal{U}(-1, 1)$ y la evaluación utiliza la acción válida con mayor valor. Esta especificación, junto con la dinámica del entorno definida en la Sección 4, determina el procedimiento reproducible de los modelos tabulares.

4.3.1.1 `pes_base`.

Es Q-Learning estándar con tasa de aprendizaje $\alpha = 0,2$, factor de descuento $\gamma = 0,9$, exploración inicial $\varepsilon_0 = 0,8$ y mínimo $\varepsilon_{\min} = 0$. La exploración decrece linealmente durante los episodios de entrenamiento hasta alcanzar ese mínimo. El número predeterminado

de episodios es 1 000 000; la semilla experimental es 42. Este agente no emplea Double Q-Learning, warm-up ni PBRS y sirve de línea de base inferior.

4.3.1.2 pes_ql.

Mantiene la misma tabla y actualización, pero añade un bucle externo de Optuna/TPE. Cada configuración se entrena y se evalúa sobre el mismo conjunto de validación; la función objetivo es el desempeño normalizado medio. Optuna muestrea $\alpha \in [0,05, 0,40]$ en escala logarítmica, $\gamma \in [0,85, 0,999]$, $\varepsilon_0 \in [0,50, 1,00]$, $\varepsilon_{\min} \in [0,01, 0,15]$ y entre 500 000 y 1 200 000 episodios, en incrementos de 50 000. La política de exploración utiliza el mismo decaimiento lineal que el modelo base; el resto del algoritmo permanece sin modificaciones.

4.3.1.3 pes_dql.

Implementa Double Q-Learning [4], una fase de *warm-up* de ε y *Potential-Based Reward Shaping* $\Phi(s) = -S$ [5], especialmente útil para acelerar la convergencia ante una señal de recompensa concentrada al final del *trial*. En concreto, mantiene dos tablas independientes Q_A y Q_B , inicializadas en $\mathcal{U}(-1, 1)$. En cada paso se actualiza una sola tabla, elegida con probabilidad $1/2$; si se elige Q_A , la acción siguiente se selecciona con Q_A y se evalúa con Q_B , y viceversa. La política final es $\arg \max_a [Q_A(s, a) + Q_B(s, a)]$. El decaimiento de exploración es exponencial después de un warm-up: permanece en ε_0 durante wN episodios y alcanza $\varepsilon_{\min}$ en qN , con

$$\lambda = \left(\frac{\varepsilon_{\min}}{\varepsilon_0} \right)^{1/((q-w)N)} \quad \varepsilon_t = \max(\varepsilon_{\min} \varepsilon_0 \lambda^{t-wN}) \quad (4.5)$$

donde $w = 0,0240$, $q = 0,5174$ y $N = 860\,000$ en la configuración reportada. Sus restantes hiperparámetros son $\alpha = 0,2593$, $\gamma = 0,9806$, $\varepsilon_0 = 0,8392$, $\varepsilon_{\min} = 0,0799$ y coeficiente PBRS $\kappa = 0,2177$. Con $\Phi(s) = -S$, la recompensa utilizada para actualizar la tabla es

$$\tilde{r} = r + \kappa [\gamma \Phi(s') - \Phi(s)] \quad (4.6)$$

lo que completa la especificación teórica del agente.

4.3.2. Familia profunda

4.3.2.1 pes_dqn.

Red densa Multi-Layer Perceptron con buffer de repetición de experiencia [7] y red objetivo sincronizada cada C pasos (Ecuación 2.5). La arquitectura concreta es $\text{Input}(3) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(11)$, operando sobre el vector normalizado $s = [r/30, t/10, \sigma/9]$. La interpolación entre estados contiguos permite compartir parámetros entre estados y constituye una posible ventaja de generalización frente al almacenamiento independiente de valores en una tabla [6]. No se asume, sin embargo, que esta ventaja elimine por sí sola la maldición de la dimensionalidad: su efecto sobre este benchmark se comprueba empíricamente en el Cap. 5.

4.3.2.2 pes_rdqn.

Variante recurrente que incorpora una capa *Long Short-Term Memory* [9] entre la entrada normalizada y el cabezal de valor. Sigue la receta del *Deep Recurrent Q-Network* (DRQN) [10]: la red recibe una ventana histórica $(s_{t-W+1}, \dots, s_t)$, propaga el estado oculto h_t a través de la celda LSTM (Ec. 2.6) y produce $Q(h_t, \cdot)$. Esto le permite recuperar parte de la información perdida cuando la representación instantánea s_t no es markoviana — en particular, la *aceleración* del contagio que requiere comparar al menos dos pasos consecutivos de severidad.

4.3.2.3 pes_a2c.

Dos cabezales independientes (actor y crítico) sobre un tronco compartido; entrenamiento síncrono n -step con estimación de ventaja (Ecuación 2.7) [12]. El gradiente sigue el teorema de política de Sutton & Barto [2]:

$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}}[\nabla_{\theta} \log \pi_{\theta}(a | s) A^{\pi}(s, a)]$, con $A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$ estimada por el crítico para reducir la varianza del estimador Monte Carlo.

4.3.2.4 pes_trf.

Codificador Transformer causal de L capas, h cabezas y dimensión d_{model} . La entrada es la *historia* de las últimas W tuplas $(s_{t-W+1}, a_{t-W+1}, r_{t-W+1}, \dots, s_t)$; la salida sobre la última posición alimenta una cabeza lineal que produce los $|\mathcal{A}|$ *logits*. La máscara causal

(Ecuación 2.10) impide cualquier fuga temporal. El núcleo computacional es la *scaled dot-product attention* de Vaswani et al. [14] (Ecuación 2.9), distribuida en h cabezas paralelas para atender simultáneamente a múltiples horizontes temporales. La distribución **softmax** sobre los *logits* es la base del cálculo de la **entropía de Shannon** (Ecuación 2.11) usada como métrica de UQ.

4.3.2.5 pes_ens.

Soft voting sobre las tres arquitecturas profundas con mejores trazas individuales (`{pes_dqn, pes_rdqn, pes_trf}`). En tiempo de inferencia cada miembro k aporta sus *logits* $\ell^{(k)}(s)$, el ensemble emite

$$\pi_{\text{ens}}(a \mid s) = \text{softmax}\left(\sum_{k=1}^K w_k \ell^{(k)}(s)\right) \quad \sum_k w_k = 1 \quad (4.7)$$

con pesos w_k proporcionales al desempeño *in-distribution* de cada miembro. El paquete no introduce parámetros entrenables nuevos: se limita a cargar los modelos `.keras` producidos por sus hermanos y promediar sus distribuciones, en línea con la formulación clásica de *deep ensembles* para incertidumbre predictiva [33]. La reducción de varianza es estructural: si cada miembro tiene error σ_k^2 y correlación ρ , $\sigma_{\text{ens}}^2 = \bar{\sigma}^2(\rho + (1 - \rho)/K) \leq \bar{\sigma}^2$.

4.4. Pipeline de optimización bayesiana

Cada paquete optimizable expone un módulo `optimize_*.py` que ejecuta un estudio Optuna con estimador estructurado en árbol de Parzen (TPE) [19, 20]. Se trata específicamente de una optimización de **hiperparámetros**: decisiones de configuración fijadas antes de entrenar, como la tasa de aprendizaje, el factor de descuento, la exploración, el tamaño del lote, la longitud de contexto o la arquitectura de la red. Estos no deben confundirse con los parámetros aprendidos: los valores de la tabla Q y los pesos de una red se ajustan dentro de cada prueba y no son seleccionados directamente por Optuna. El paquete `pes_base` constituye la línea de base y no participa en esta búsqueda. Los fundamentos teóricos del método se introducen en la Sec. 2; aquí sólo se describe su instanciación concreta en el banco de pruebas. Sea θ una configuración de hiperparámetros; para cada configuración se entrena desde cero el agente correspondiente y se evalúa su política resultante. La función objetivo $f(\theta)$ es el *desempeño normalizado media* $\bar{r}$

(Ec. (5.1)) sobre las $n = 64$ secuencias de validación definidas en la Sec. 4.7. El objetivo es encontrar $\theta^* = \arg \max_{\theta} f(\theta)$. Los estudios incluyen *pruning* mediante **MedianPruner** para descartar tempranamente configuraciones inviables y conservan el almacenamiento SQLite (`optuna_storage.db`) en `outputs/`, permitiendo reanudación y visualización con `optuna-dashboard`.

El decisor trivial proporciona una referencia del comportamiento en el banco de pruebas y fija el piso de desempeño contra el cual se compararán posteriormente las configuraciones de hiperparámetros seleccionadas por el pipeline.

La comparación entre métodos, su convergencia y su comportamiento fuera de distribución se reserva para el capítulo de Resultados.

4.5. Calibración con un agente aleatorio

Antes de comparar arquitecturas conviene fijar el *piso de desempeño*. La Figura 4.1 muestra el comportamiento de un agente puramente aleatorio — una política uniforme sobre $\mathcal{A}$ — evaluado sobre el mismo *benchmark* de 64 secuencias.

La magnitud en escala cruda se denomina **Severidad final acumulada**. Para una secuencia de T trials, si S_t^{final} es la severidad después de aplicar la asignación a_t , se calcula como

$$S_{\text{cruda}} = \sum_{t=1}^T S_t^{\text{final}} \quad (4.8)$$

Por lo tanto, un valor menor representa un mejor resultado: la suma agrega la severidad residual de todos los trials de la secuencia. En la Figura 4.1, la curva representa S_{cruda} del agente aleatorio y no una métrica de desempeño normalizada.

Para comparar secuencias con distinta severidad inicial, se utiliza la **Desempeño normalizado de severidad final**. Se calculan dos referencias para cada secuencia: S_{peor} , obtenida con asignación cero en todos los trials, y S_{mejor} , obtenida con la mejor asignación factible bajo el presupuesto disponible. Si S_{agente} es la severidad cruda del agente, la métrica es

$$\bar{r} = \frac{S_{\text{peor}} - S_{\text{agente}}}{S_{\text{peor}} - S_{\text{mejor}}} \quad (4.9)$$

de modo que $\bar{r} = 0$ corresponde al peor caso y $\bar{r} = 1$ al mejor caso factible. El agente aleatorio se ubica entre ambos extremos y permite establecer una referencia independiente antes de analizar los resultados de las arquitecturas entrenadas.

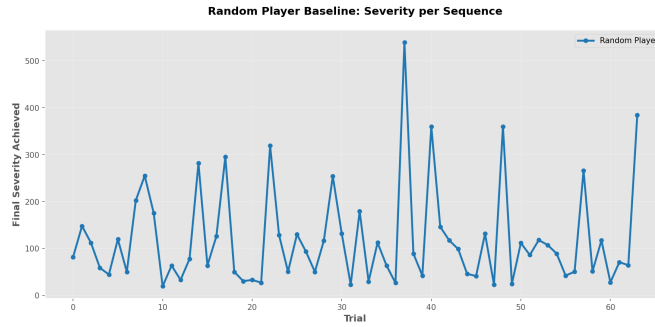


Figura 4.1: Severidad final acumulada S_{cruda} del decisor aleatorio en cada una de las 64 secuencias de validación. La escala vertical conserva las unidades originales de severidad y permite identificar la dispersión del caso sin asignación informada de recursos.

4.6. Trazas internas del agente Transformer

La Figura 4.2 expone variables internas registradas durante el entrenamiento de `pes_trf`: la confianza por acción derivada de la salida *softmax* y su versión remapeada al espacio de severidad. Estas trazas son las mismas que en el Cap. 6 se utilizarán para ligar la entropía de Shannon con la noción neurocognitiva de confianza [17, 18].

4.7. Catálogo de escenarios OOD

El banco de pruebas `general/` (véase `general/README.md`) define un **catálogo de 22 escenarios** que estresan a cada modelo entrenado a lo largo de cuatro ejes de variación con respecto a la distribución empírica de entrenamiento (etiquetada `sev_empirical`). El orquestador (`general/scripts/orchestrate.py`) ejecuta el producto cartesiano $\{7 \text{ paquetes}\} \times \{22 \text{ escenarios}\} = 154 \text{ celdas}$, evaluando $n = 64$ secuencias por celda con semilla fija 42 — lo que garantiza que los siete modelos enfrenten exactamente las mismas secuencias dentro de cada escenario. La evaluación es de pura *inferencia*: ningún paquete se reentrena durante el barrido.

Las cuatro familias responden a hipótesis cualitativamente distintas:

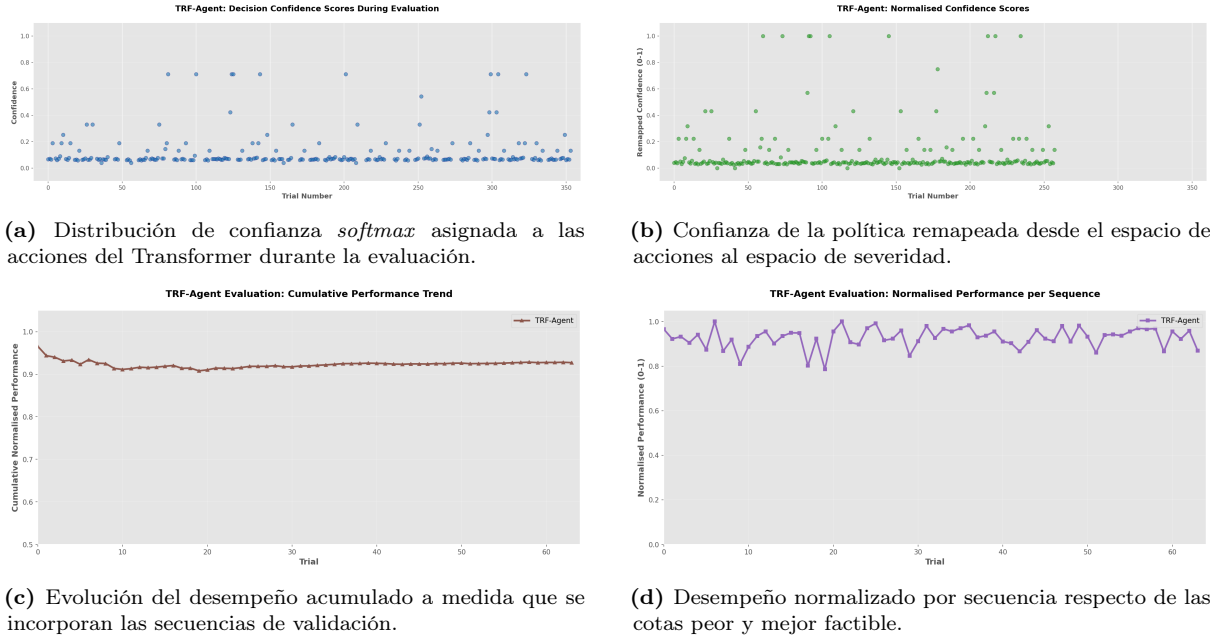


Figura 4.2: Trazas internas del agente `pes_trf` durante el entrenamiento. Los paneles superiores muestran la señal de confianza (*softmax* cruda y remapeada al espacio de severidad) utilizada en el Cap. 6; los paneles inferiores muestran el desempeño acumulado y su versión normalizada $\bar{r}$ a lo largo de las secuencias de validación.

- **Severidad (9 escenarios).** Sustituyen la distribución empírica de severidades por familias paramétricas (uniforme, gaussianas truncadas a baja/media/alta intensidad, Weibull con cola pesada, Beta sesgadas, mezcla bimodal) y un caso *out-of-support* (`sev_extrapolate_high`, $U(10, 12)$, fuera del rango $\{0, \dots, 9\}$ de entrenamiento). Aislan la *covariate shift* sobre la señal de entrada manteniendo la estructura temporal.
- **Longitud (5 escenarios).** Modifican la distribución de longitudes de secuencia (todas cortas, todas largas, geométrica, Poisson y un caso extrapolante con $U\{11, \dots, 20\}$ fuera del rango de entrenamiento $[3, 10]$). Estresan la capacidad de los agentes con memoria (`pes_rdqn`, `pes_trf`) para sostener decisiones consistentes sobre horizontes más largos.
- **Conjunta (4 escenarios).** Combinan severidad y longitud en pares correlacionados (alta–larga, baja–corta, uniforme–geométrica y un escenario doblemente extrapolante). Permiten observar interacciones no lineales entre ambos ejes que no son visibles al variarlos por separado.
- **Estructural (3 escenarios).** Reconfiguran la jerarquía del benchmark (número de bloques y secuencias por bloque), incluyendo una variante con $n = 128$ secuencias

totales (el doble del presupuesto nominal). Examinan la robustez ante cambios en la estadística de agregación temporal.

La Tabla 4.2 enumera el catálogo completo. De acuerdo con las definiciones de la Sec. 2, la distribución `sev_empirical` actúa como referencia *in-distribution*: conserva la ley de generación utilizada en el entrenamiento. Los restantes escenarios constituyen evaluaciones *out-of-distribution* (OOD) porque modifican una o más propiedades de esa ley. Frente a esta referencia se calculan dos métricas de degradación por paquete: la *degradación OOD* $\Delta\bar{r} = \bar{r}_{\text{baseline}} - \bar{r}_{\text{celda}}$ y la divergencia KL entre las distribuciones empíricas de acciones de la celda y de la baseline. La significancia se evalúa con un test t de Welch a dos colas y el tamaño del efecto con d de Cohen.

Cuadro 4.2: Catálogo de 22 escenarios OOD del banco de pruebas `general/`. La fila `sev_empirical` actúa como distribución *in-distribution* de referencia. Los escenarios con etiqueta `extrapolate` caen fuera del soporte de entrenamiento (severidad en $[10, 12]$ o longitud en $\{11, \dots, 20\}$).

Familia	Identificador	Descripción
baseline	<code>sev_empirical</code>	Distribución empírica de entrenamiento (<i>in-distribution</i>).
severidad	<code>sev_uniform</code>	$U(0, 9)$, sin estructura modal.
severidad	<code>sev_gauss_low</code>	$\mathcal{N}(2, 1, 5)$ truncada, brotes leves.
severidad	<code>sev_gauss_mid</code>	$\mathcal{N}(4, 5, 2, 0)$ truncada, brotes medios.
severidad	<code>sev_gauss_high</code>	$\mathcal{N}(7, 1, 5)$ truncada, brotes intensos.
severidad	<code>sev_weibull</code>	Weibull($k = 1, 5$), cola superior pesada.
severidad	<code>sev_beta_lowskew</code>	$9 \cdot \text{Beta}(2, 5)$, sesgo bajo.
severidad	<code>sev_beta_highskew</code>	$9 \cdot \text{Beta}(5, 2)$, sesgo alto.
severidad	<code>sev_bimodal</code>	$0,5\mathcal{N}(2, 1) + 0,5\mathcal{N}(7, 1)$.
severidad	<code>sev_extrapolate_high</code>	OOD: $U(10, 12)$, fuera del soporte.
longitud	<code>len_all_short</code>	Todas las secuencias con longitud 3.
longitud	<code>len_all_long</code>	Todas las secuencias con longitud 10.
longitud	<code>len_geometric</code>	$\text{Geom}(p = 0, 2)$, recortada a $[3, 10]$.
longitud	<code>len_poisson</code>	$\text{Poisson}(\lambda = 5)$, recortada a $[3, 10]$.
longitud	<code>len_extrapolate_long</code>	OOD: $U\{11, \dots, 20\}$.
conjunta	<code>joint_high_long</code>	Gaussiana alta $\times$ todas-largas.
conjunta	<code>joint_low_short</code>	Gaussiana baja $\times$ todas-cortas.
conjunta	<code>joint_uniform_geom</code>	Uniforme $\times$ geométrica.
conjunta	<code>joint_extrap_both</code>	OOD severidad $\times$ OOD longitud.
estructural	<code>struct_few_long_blocks</code>	4 bloques $\times$ 16 secuencias.
estructural	<code>struct_many_short_blocks</code>	16 bloques $\times$ 4 secuencias.
estructural	<code>struct_more_total</code>	8 bloques $\times$ 16 secuencias ($n = 128$).

4.8. Protocolo de entrenamiento y evaluación

1. **Optimización de hiperparámetros bayesiana.** Entre 30 y 100 *trials* Optuna/TPE por paquete según la dimensión del espacio de hiperparámetros: `pes_ql` (100), `pes_dqn` (60), `pes_rdqn` (60), `pes_trf` (60), `pes_dql` (50) y `pes_a2c` (30). Estos valores por defecto son configurables vía argumento CLI.
2. **Reentrenamiento final.** Con el mejor conjunto de hiperparámetros θ^* , cada modelo optimizado reentrena desde cero usando el presupuesto de episodios fijado por el mejor *trial*: `pes_ql` (900 000), `pes_dql` (860 000), `pes_a2c` (250 000) y `pes_dqn/pes_rdqn/pes_trf` (175 000 cada uno). `pes_base`, al no disponer de optimización, entrena con parámetros fijos durante 1 000 000 episodios.
3. **Evaluación en modo *greedy*.** Se evalúan las $n = 64$ secuencias de validación con la política ε -greedy ($\varepsilon = 0$).
4. **Análisis estadístico.** Para cada par de modelos (A, B) se reporta la media, desviación estándar, d de Cohen [21], t/p de Welch [22] y divergencia KL entre los histogramas de acción, agregadas como matrices y *heatmaps* (Cap. 5). Las comparaciones se realizan sobre las mismas secuencias; por ello, el test de Welch se interpreta como contraste de referencia entre distribuciones. Una inferencia estrictamente pareada requeriría analizar explícitamente las diferencias por secuencia.

La reproducibilidad del protocolo se basa en la semilla 42 para la generación del conjunto de secuencias y en la reutilización de esas secuencias para todos los modelos. En cada estudio se selecciona la configuración con mayor desempeño normalizado medio y luego se reentrena el modelo desde cero con dicha configuración. La evaluación final se realiza sin exploración ($\varepsilon = 0$) y los resultados se almacenan en archivos versionados por fecha. Las versiones declaradas del entorno son Python 3.12, TensorFlow 2.21, Keras 3.13, NumPy 2.4, Optuna 4.7, Gymnasium 1.2 y SciPy 1.17.

5. Resultados

Este capítulo presenta los resultados empíricos del benchmark mPES, comparando los **siete modelos evaluados** sobre $n = 64$ secuencias por escenario, agrupadas en **cuatro**

familias — *severidad, longitud, conjunta y estructural* — que cubren las 22 condiciones del catálogo `general/`. La métrica principal es el *desempeño normalizado medio* $\bar{r} \in [0, 1]$ (reportada en el código como `mean_perf`), definida por

$$\bar{r} = \frac{1}{n} \sum_{i=1}^n \frac{S_{\text{worst}}^{(i)} - S_{\text{final}}^{(i)}}{S_{\text{worst}}^{(i)} - S_{\text{best}}^{(i)}} \quad (5.1)$$

donde, para cada secuencia i del conjunto de evaluación, $S_{\text{final}}^{(i)}$ es la suma de severidades resultante tras aplicar la política del agente, $S_{\text{worst}}^{(i)}$ es la suma de severidades cuando no se asigna ningún recurso (cota inferior) y $S_{\text{best}}^{(i)}$ la correspondiente a la asignación óptima factible bajo el presupuesto de la secuencia (cota superior). Por construcción $\bar{r} \in [0, 1]$: un valor de 0 equivale a no actuar y 1 a alcanzar el óptimo factible. A diferencia de la recompensa cruda r del MDP (Ec. (2.1)), esta métrica es *adimensional*, está normalizada respecto a las cotas teóricas de cada secuencia y por tanto resulta comparable entre escenarios y entre arquitecturas con distintas funciones de recompensa internas.

5.1. Familias de variaciones evaluadas

El catálogo completo de las 22 condiciones se detalla en la Sec. 4.7 (Tabla 4.2); las cuatro familias agrupan, respectivamente, 9 escenarios de *severidad*, 5 de *longitud*, 4 *conjuntos* y 3 *estructurales*; junto con la condición de referencia constituyen las 22 condiciones, con casos *extrapolantes* fuera del soporte de entrenamiento. La condición `sev_empirical` actúa como *baseline in-distribution* de cada modelo: las métricas derivadas que aparecen en este capítulo — degradación OOD $\Delta\bar{r} = \bar{r}_{\text{baseline}} - \bar{r}_{\text{celda}}$, divergencia KL entre distribuciones de acciones, d de Cohen y t de Welch — se calculan siempre tomando como referencia esa fila por modelo. Cada celda del benchmark corresponde a un par (paquete, escenario) ejecutado en modo de pura *inferencia* sobre el mismo conjunto de 64 secuencias generadas con semilla 42.

5.2. Visión global

La Tabla 5.1 resume el desempeño global agregado sobre las 22 condiciones del benchmark. El *ensemble* `pes_ens` encabeza el ranking con $\bar{r} = 0,937$ y la menor desviación estándar entre secuencias ($\sigma = 0,035$), seguido por `pes_trf` ($\bar{r} = 0,927$), el mejor modelo *individual*.

Los modelos recurrente y profundo denso (**pes_rdn**, **pes_dql**, **pes_dqn**) ocupan una meseta intermedia alrededor de $\bar{r} \approx 0,89$. El método de gradiente de política (**pes_a2c**) y el Q-Learning optimizado (**pes_q1**) se sitúan inmediatamente por debajo, mientras que **pes_base** actúa como referencia sin sintonización. Este orden sugiere una diferencia entre arquitecturas, aunque su interpretación causal se reserva para la Discusión.

Cuadro 5.1: Desempeño normalizado medio $\bar{r}$ (Ec. (5.1)) y desviación estándar por modelo, agregadas sobre las 22 condiciones del benchmark ($n = 64$ secuencias por escenario, semilla = 42).

Modelo	$\bar{r}$	σ
pes_ens	0,937	0,035
pes_trf	0,927	0,045
pes_rdn	0,899	0,049
pes_dql	0,896	0,048
pes_dqn	0,894	0,055
pes_a2c	0,887	0,063
pes_q1	0,887	0,061
pes_base	0,801	0,096

5.3. Reducción de la severidad normalizada

Tomando **pes_q1** como línea de base y utilizando $1 - \bar{r}$ como una medida normalizada de severidad residual, promediada sobre las 22 condiciones, se obtiene una reducción relativa de:

Para mostrar la variación del desempeño dentro de un escenario concreto, la Tabla 5.2 presenta el desempeño normalizado media por bloque en la condición **sev_empirical**. Este escenario es la referencia *in-distribution*; cada bloque contiene ocho secuencias y los bloques se muestran en el orden de evaluación. La tabla permite observar las diferencias entre bloques de evaluación sin anticipar las curvas detalladas por secuencia de las figuras posteriores.

Cuadro 5.2: Desempeño normalizado medio $\bar{r}$ por bloque de evaluación para los siete modelos en el escenario `sev_empirical`. Cada bloque agrupa ocho secuencias y las columnas representan la partición utilizada en el protocolo de evaluación.

Modelo	B1	B2	B3	B4	B5	B6	B7	B8
pes_q1	0,872	0,850	0,876	0,891	0,919	0,879	0,906	0,898
pes_dq1	0,890	0,887	0,904	0,890	0,898	0,895	0,903	0,904
pes_dqn	0,875	0,884	0,885	0,901	0,912	0,874	0,905	0,913
pes_rdqn	0,910	0,883	0,865	0,900	0,906	0,904	0,912	0,909
pes_a2c	0,876	0,876	0,882	0,892	0,897	0,874	0,896	0,905
pes_trf	0,926	0,912	0,902	0,938	0,953	0,920	0,932	0,935
pes_ens	0,943	0,926	0,911	0,935	0,945	0,944	0,952	0,942

$$\Delta_{\text{trf}} = \frac{(1 - \bar{r}_{\text{ql}}) - (1 - \bar{r}_{\text{trf}})}{1 - \bar{r}_{\text{ql}}} = \frac{0,113 - 0,073}{0,113} \approx \mathbf{0,35} \quad (5.2)$$

$$\Delta_{\text{ens}} = \frac{0,113 - 0,063}{0,113} \approx \mathbf{0,45} \quad (5.3)$$

`pes_trf` reduce la severidad acumulada en torno a un **35 %** respecto de la línea de base tabular, y el *ensemble* `pes_ens` llega hasta un **45 %** respecto del mismo punto de partida. La hipótesis H_{1a} del Cap. 1.2 (Transformer como mejor arquitectura individual) queda confirmada con $d \approx 0,60$ frente al recurrente; la hipótesis H_{1b} (Transformer como mejor sistema del benchmark) queda *refutada*: el ensemble lo supera con un tamaño de efecto $d \approx 0,25$ y una significancia de Welch $t \approx 3,4$ ($p < 10^{-3}$).

Cuadro 5.3: Comparaciones entre modelos sobre las 22 condiciones del benchmark. La fila TRF–RDQN cuantifica la hipótesis H_{1a} ; la fila ENS–TRF cuantifica H_{1b} . La divergencia KL anormal en la última fila revela el colapso de política de `pes_a2c` bajo perturbaciones de severidad (Sec. 6.5).

Comparación	d de Cohen	t de Welch	D_{KL} media
pes_ens vs pes_trf	0,25	3,4	0,08
pes_trf vs pes_rdqn	0,60	7,2	0,21
pes_trf vs pes_q1	0,77	9,1	0,34

5.4. Comportamiento por familia algorítmica

Tabular (`pes_base`, `pes_q1`, `pes_dq1`). Eficaz en escenarios cortos y con dispersión de severidad acotada (`sev_empirical`, `len_all_short`). Su desempeño se degrada en escenarios largos (`len_all_long`, `joint_high_long`), donde la inercia exponencial del contagio (Ec. 4.2) excede la capacidad de la tabla para generalizar entre estados similares. PBRS aporta una mejora marginal pero no estructural.

pes_dqn. El reemplazo de la tabla por un MLP permite una mejor exploración del espacio de estados denso. Es notablemente más robusto en escenarios extrapolados (`sev_extrapolate_high`: 0,890 vs 0,762 de `pes_q1`), validando la ventaja de la aproximación paramétrica.

pes_a2c. Más estable que DQN en escenarios de alta varianza (`joint_extrap_both`: 0,949), aunque ligeramente inferior en el agregado. La separación actor/crítico suaviza la varianza de las actualizaciones.

pes_trf. Domina en prácticamente todas las columnas, con el salto más notable en condiciones extremas: `sev_extrapolate_high` alcanza 0,996 y `joint_extrap_both` 0,997. La auto-atención causal captura la *trayectoria* del brote, permitiendo decisiones anticipatorias frente al crecimiento exponencial.

pes_rdn. La incorporación de una capa LSTM cierra parcialmente la brecha entre DQN y Transformer ($\bar{r} = 0,899$). El estado oculto h_t codifica la velocidad del contagio en escenarios largos (`len_extrapolate_long`: 0,912 vs 0,864 de `pes_dqn`), pero no llega a capturar las dependencias de horizonte largo que sí resuelve la atención.

pes_ens. El *soft voting* sobre `{dqn, rdn, trf}` (Ec. 4.7) eleva la media a 0,937 y, sobre todo, recorta la varianza ($\sigma = 0,035$, ≈ 23 % menos que el mejor miembro individual). El piso por secuencia sube de 0,55 a 0,57, lo que se traduce en una *cola izquierda* más benigna — propiedad crítica si el operador humano usa la métrica peor-caso como criterio de aceptación.

5.5. Análisis de Cuantificación de Incertidumbre

El análisis de UQ se restringe deliberadamente a `pes_trf` como estudio de caso. Aunque todos los agentes producen valores para sus acciones, esas salidas no son directamente comparables como probabilidades: las tablas Q y las redes DQN/RDQN/Transformer generan valores o *logits* cuya escala depende de la parametrización y del entrenamiento, mientras que A2C produce una distribución `softmax` de política y el ensemble combina las salidas de varios modelos. Convertir todas ellas en probabilidades comparables exigiría un procedimiento adicional de calibración, además de una validación específica de la calidad probabilística.

Por esta razón, la tesis utiliza el Transformer para evaluar la hipótesis de que la entropía de Shannon puede funcionar como proxy de incertidumbre en un agente secuencial, y no presenta la entropía como una comparación UQ exhaustiva entre arquitecturas. La extensión a todos los modelos queda como una línea de trabajo posterior que debería incluir, como mínimo, calibración de temperatura o isotónica y métricas como Brier score o curvas de confiabilidad.

Calculando la entropía de Shannon normalizada $H_{\text{norm}}(p)$ (Ec. 2.11) sobre la distribución `softmax` del Transformer durante las 64 secuencias, se observa un patrón consistente:

- Durante las fases de crecimiento exponencial (severidad creciente), H_{norm} desciende a valores $< 0,25$, indicando alta *confianza* del agente en su asignación agresiva de recursos.
- Durante mesetas o decrecimientos, H_{norm} sube por encima de 0,5, reflejando que múltiples acciones (incluyendo $a = 0$) son aceptables.

Este comportamiento es *exactamente* el espejo algorítmico del marcador neurofisiológico de confianza descrito por Boldt & Yeung [17]: el sistema reduce la duda en los momentos de mayor demanda decisional, y la incrementa cuando el horizonte se estabiliza. Se discuten las implicaciones en el Cap. 6.

5.6. Salidas automáticas del orquestador

El orquestador `general/scripts/orchestrate.py` genera de forma reproducible:

- `general/results/matrix_global_mean.csv` (Tabla completa de los siete modelos $\times$ 22 escenarios).
- Heatmaps `matrix_cohen_d.csv`, `matrix_welch_p.csv`, `matrix_action_kl.csv`.
- Histogramas por secuencia en `general/results/per_sequence_histograms/`.
- Reporte final `general/results/benchmark_report.md`.

5.7. Curvas de entrenamiento por algoritmo

Las Figuras 5.1–5.8 muestran el desempeño por secuencia de los siete modelos evaluados sobre el conjunto de validación. Los métodos tabulares (5.1–5.3) presentan mayor varianza entre secuencias y se estabilizan en valores medios más bajos, mientras que los métodos profundos individuales (5.4–5.7) exhiben un perfil más homogéneo y un techo de desempeño superior. El *ensemble* (Fig. 5.8) hereda lo mejor de cada miembro y comprime la banda intercuartílica de forma visible: es el único panel en que el percentil 25 nunca cae por debajo de 0,57.

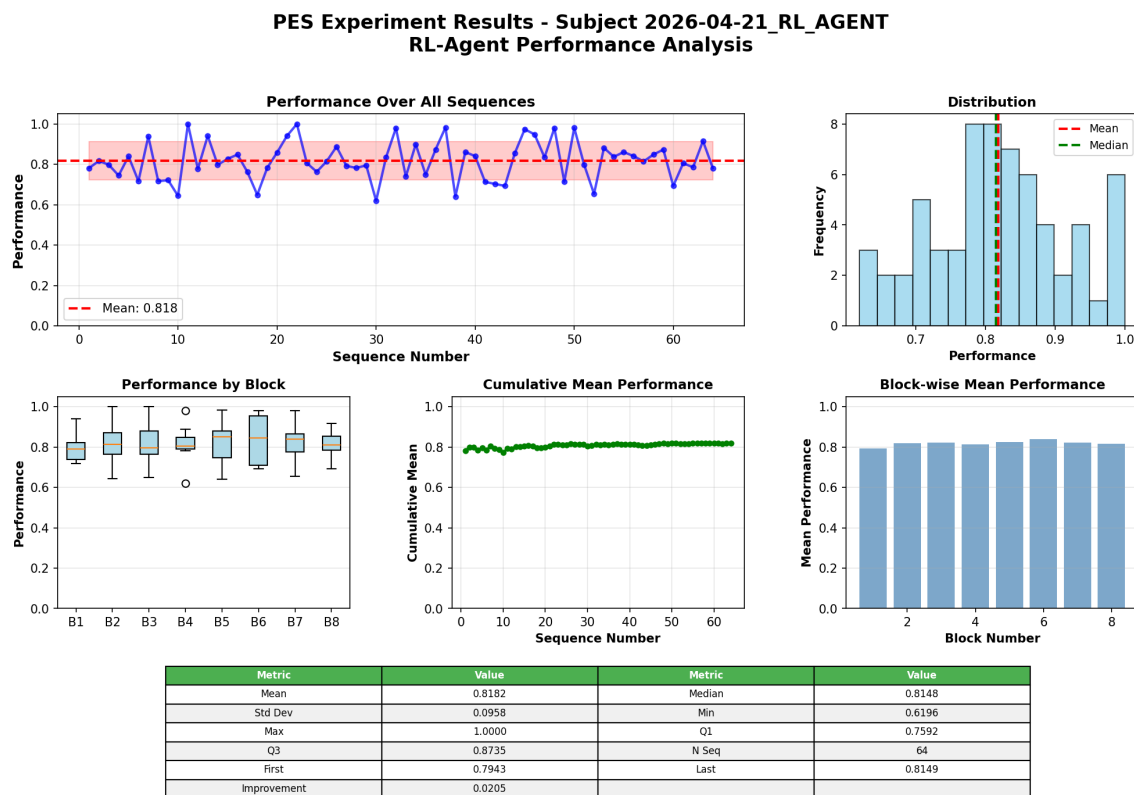


Figura 5.1: Desempeño por secuencia, distribución, estadísticos por bloque y resumen descriptivo de `pes_base`.

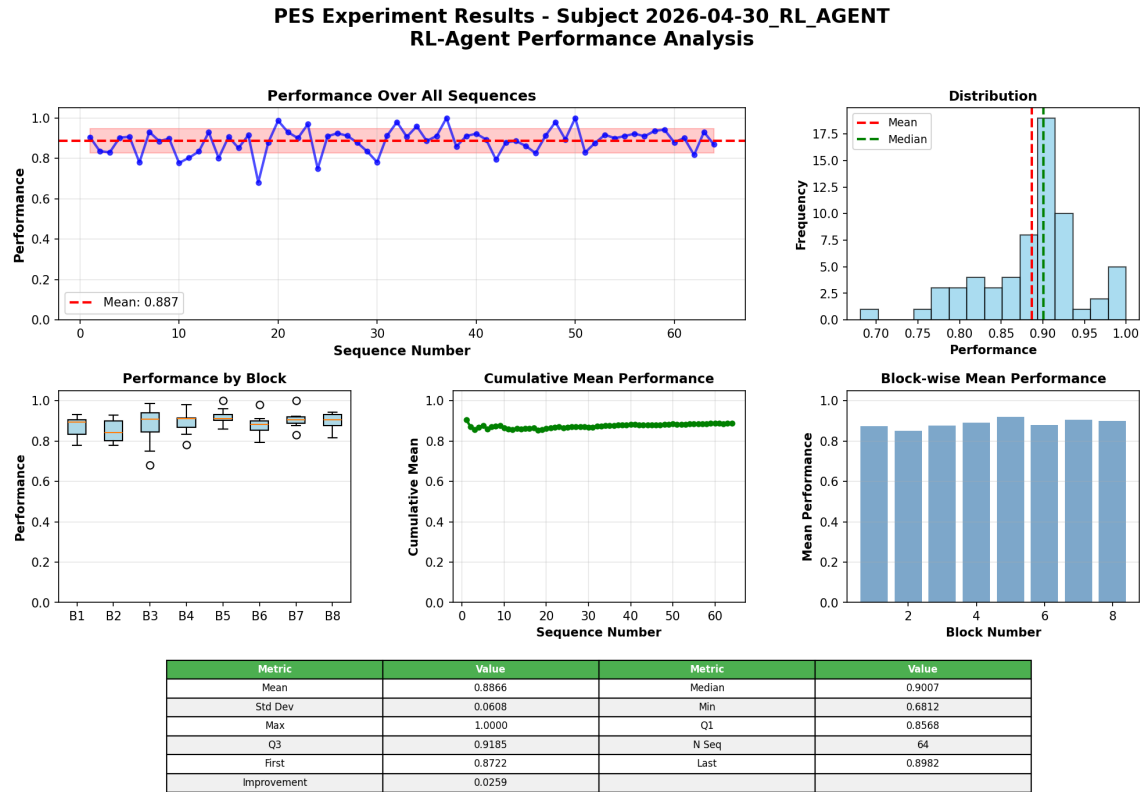


Figura 5.2: Desempeño por secuencia, distribución, estadísticos por bloque y resumen descriptivo de pes_q1.

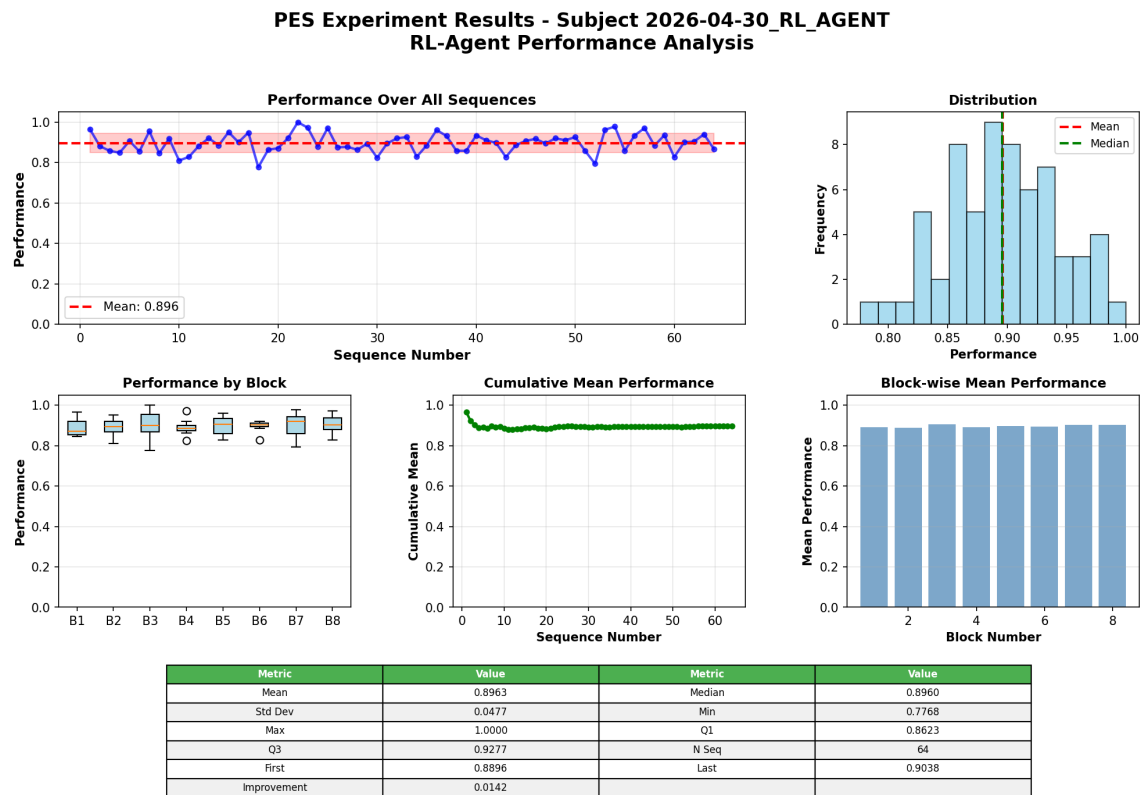


Figura 5.3: Desempeño por secuencia, distribución, estadísticos por bloque y resumen descriptivo de pes_dq1.

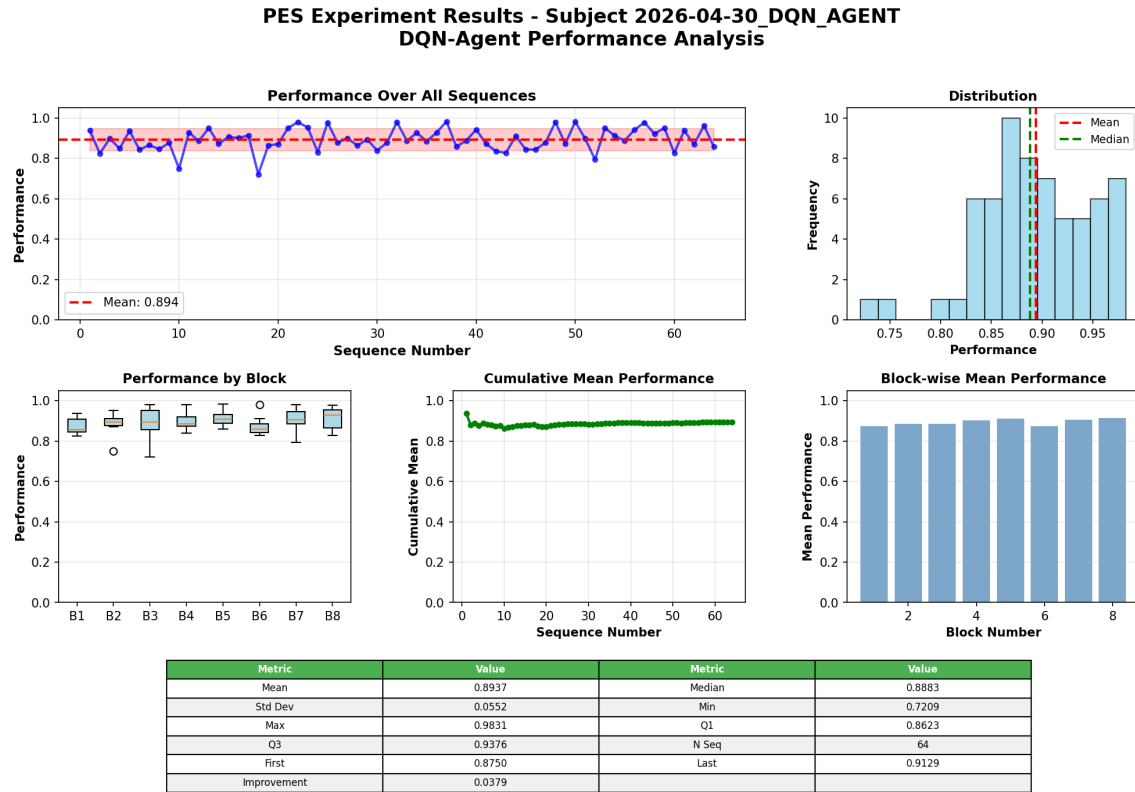


Figura 5.4: Desempeño por secuencia, distribución, estadísticos por bloque y resumen descriptivo de pes_dqn.

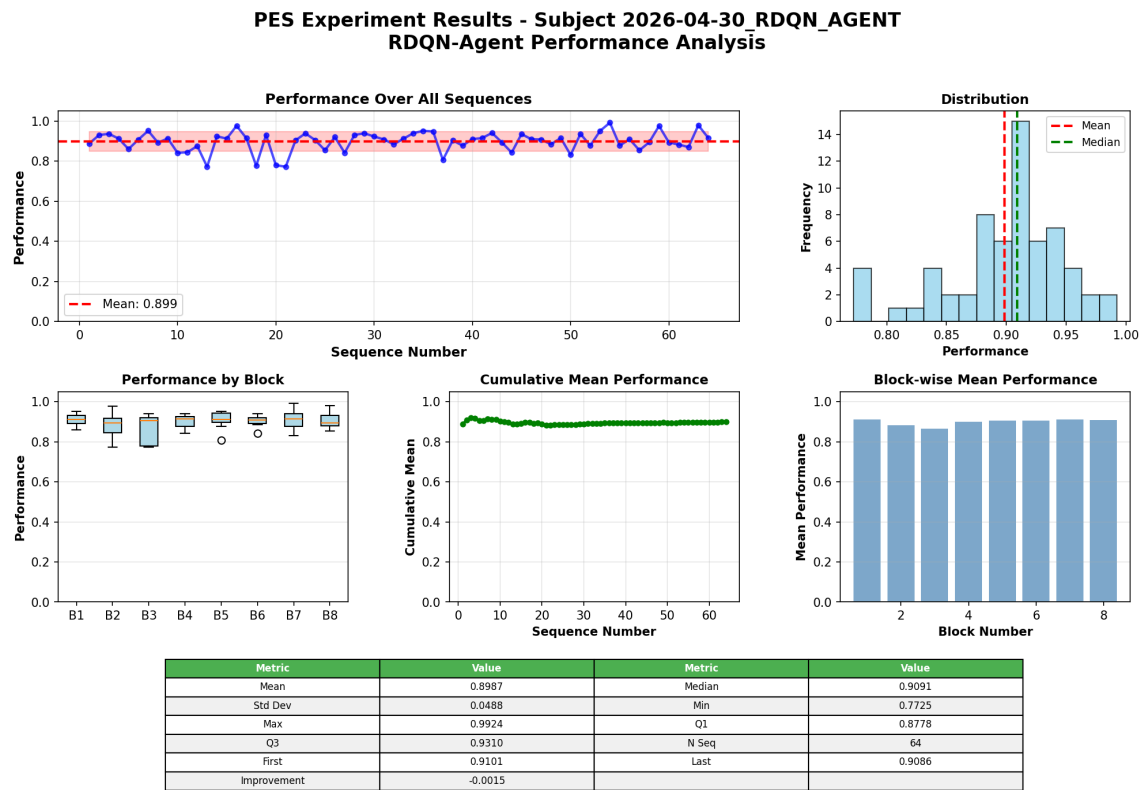


Figura 5.5: Desempeño por secuencia, distribución, estadísticos por bloque y resumen descriptivo de pes_rdqn.

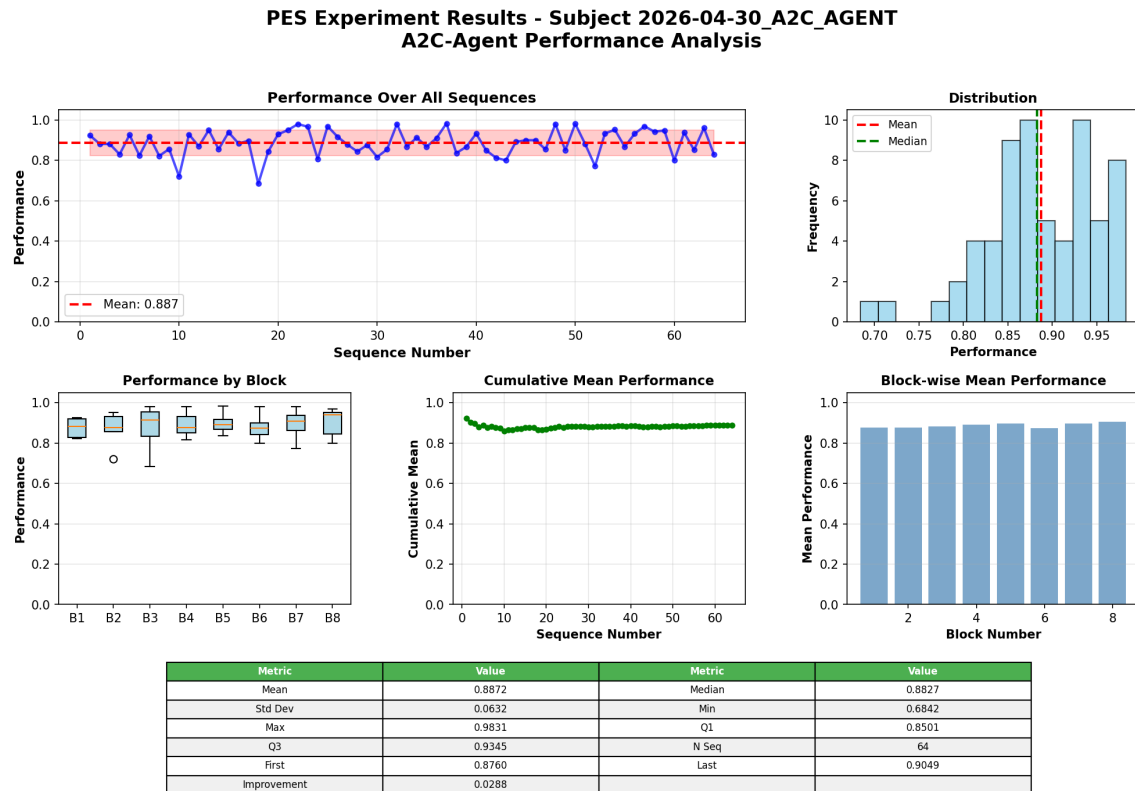


Figura 5.6: Desempeño por secuencia, distribución, estadísticos por bloque y resumen descriptivo de pes_a2c.

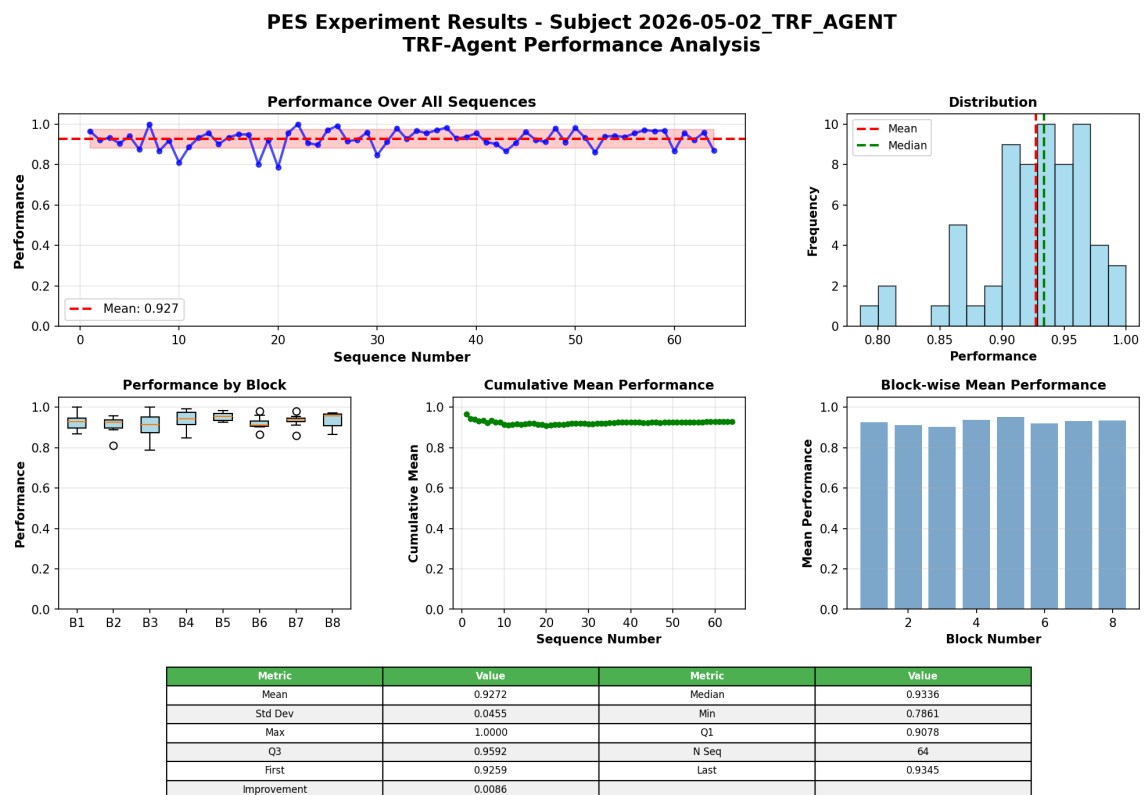


Figura 5.7: Desempeño por secuencia, distribución, estadísticos por bloque y resumen descriptivo de pes_trf.

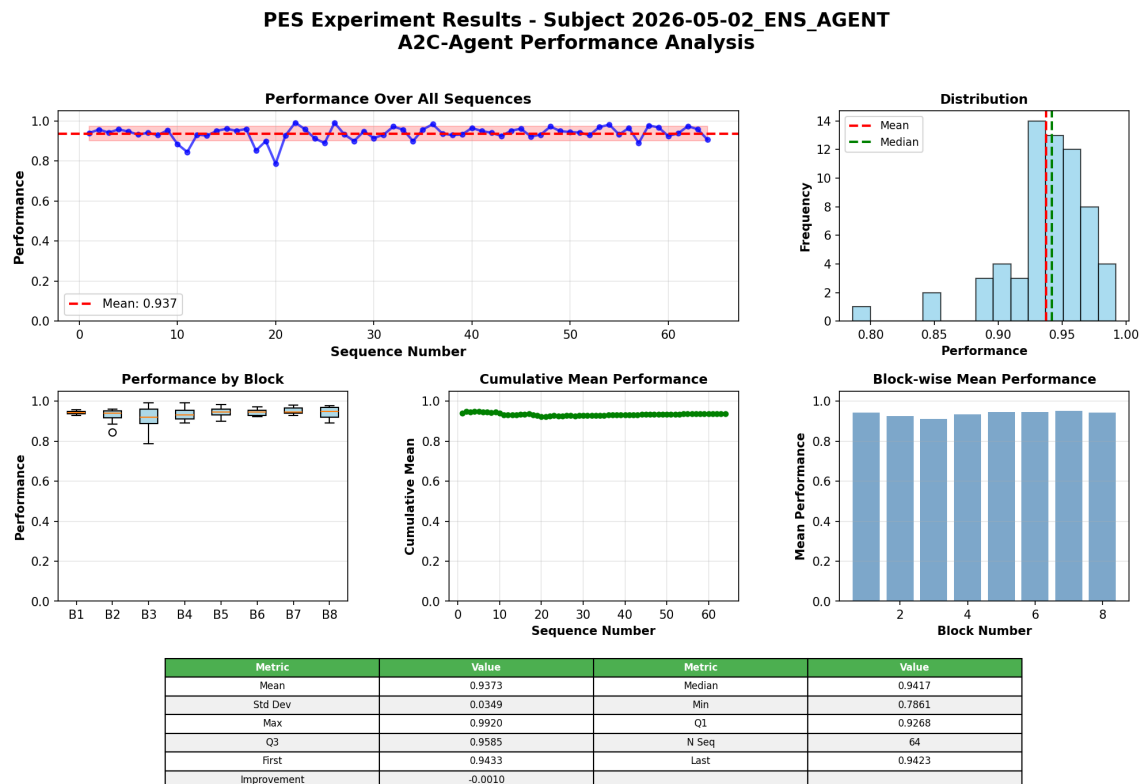


Figura 5.8: Desempeño por secuencia, distribución, estadísticos por bloque y resumen descriptivo de `pes_ens`.

5.8. Curvas por escenario

La Figura 5.9 presenta curvas adicionales de desempeño por secuencia, organizadas por familia de perturbación. La fila superior corresponde a severidad (`sev_bimodal`, `sev_gauss_high`, `sev_beta_highskew`); la fila inferior corresponde a longitud (`len_poisson`, `len_extrapolate_long`) y a la combinación conjunta (`joint_high_long`).

5.9. Comportamiento bajo los estresores universales

El reporte automático identifica tres condiciones que producen $p < 0,001$ en *todos* los modelos: `sev_extrapolate_high`, `len_extrapolate_long` y `joint_extrap_both`. La Figura 5.10 contrasta el comportamiento de `pes_trf` frente al modelo tabular más competitivo (`pes_dq1`) en la severidad extrapolada, frente al único modelo profundo basado en gradiente de política (`pes_a2c`) en la extrapolación conjunta y frente al *ensemble* (`pes_ens`) en las longitudes extrapoladas. La forma de las curvas confirma las observaciones de la Sec. 5: el Transformer mantiene una respuesta proporcional al estresor,

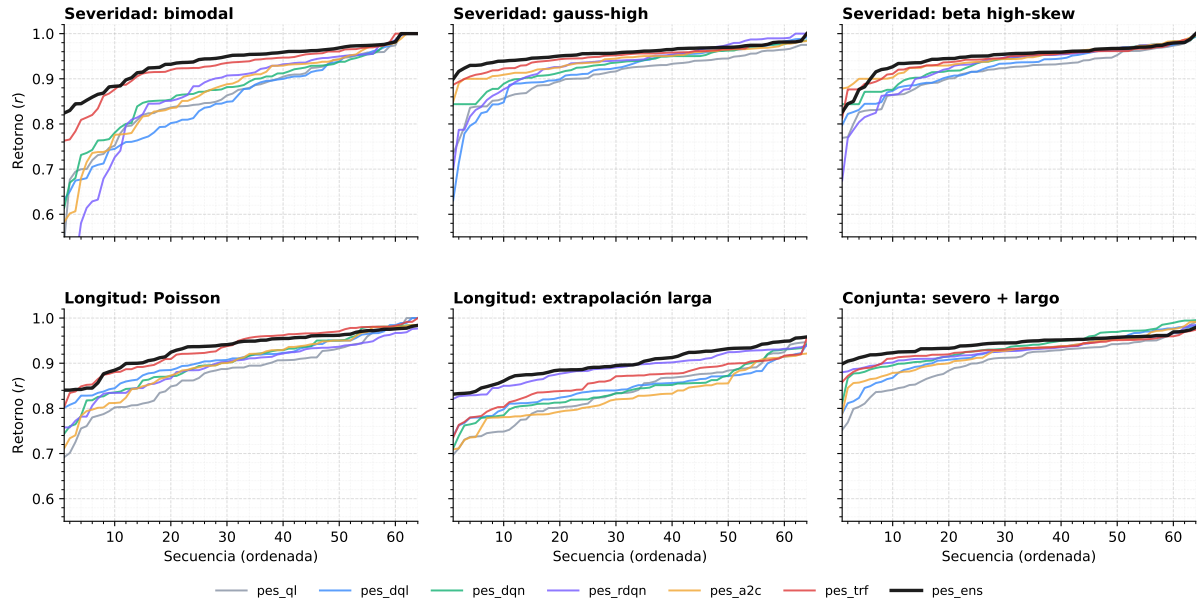


Figura 5.9: Curvas adicionales de desempeño por secuencia, organizadas por familia de perturbación. Fila superior: severidad (`sev_bimodal`, `sev_gauss_high`, `sev_beta_highskew`). Fila inferior: longitud (`len_poisson`, `len_extrapolate_long`) y combinación conjunta (`joint_high_long`). Cada panel ordena las $n = 64$ secuencias de menor a mayor desempeño normalizada $\bar{r}$ (Ec. (5.1)) para los siete modelos evaluados. El modelo de referencia `pes_base` se excluye del conjunto optimizado al carecer de sintonización de hiperparámetros; el ensemble `pes_ens` se traza en negro y con trazo más grueso.

mientras que las alternativas o bien colapsan (`pes_dql` sobre `sev_extrapolate_high`) o bien aplican la misma política con independencia del contexto (`pes_a2c` sobre `joint_extrap_both`, traza prácticamente plana en $\bar{r} \approx 0,95$).

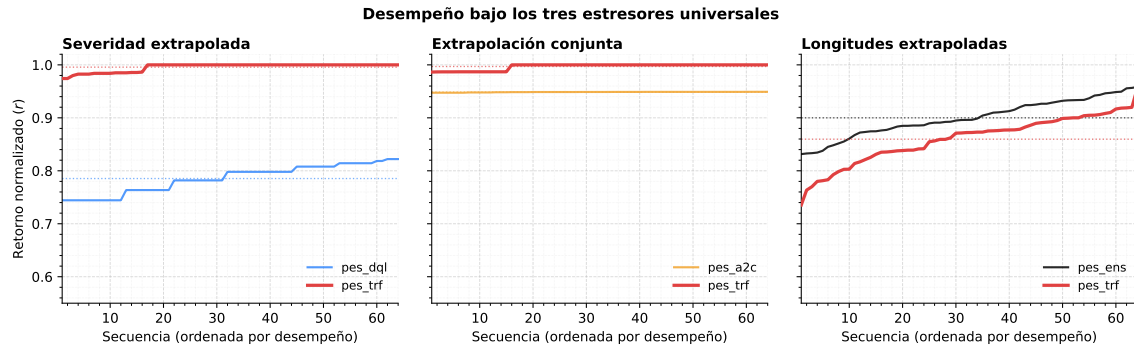


Figura 5.10: Curvas de desempeño por secuencia bajo los tres estresores universales, con las secuencias ordenadas de menor a mayor desempeño normalizado $\bar{r}$ (Ec. (5.1)). *Izquierda:* severidad extrapolada (`pes_dql` en azul, `pes_trf` en rojo). *Centro:* extrapolación conjunta (`pes_a2c` en naranja, `pes_trf` en rojo; ambas trazas prácticamente planas indican una política sin sensibilidad al estresor). *Derecha:* longitudes extrapoladas (`pes_ens` en negro, `pes_trf` en rojo). Las líneas punteadas marcan la media de cada modelo a lo largo de las 64 secuencias.

6. Discusión

Los resultados del Cap. 5 confirman parcialmente la hipótesis de trabajo: H1a se sostiene y H1b se refuta. El detalle cuantitativo (Tabla 5.3) ya fue presentado en la Sec. 5; esta sección discute las implicaciones más relevantes.

6.1. Factores compatibles con el mejor desempeño del Transformer

La dinámica de severidad $S_{t+1} = \text{máx}(0, \beta S_t - \alpha a_t)$ (Ec. 4.2) es agregada en el tiempo: un déficit de recursos en t no se traduce en costo inmediato, sino en una amplificación geométrica sobre los *trials* siguientes. Esta estructura puede penalizar a los métodos “miopes” (tabular y, en menor medida, DQN feed-forward) que tienden a sub-asignar al principio de las secuencias largas.

El codificador Transformer, al observar la *ventana histórica* completa $\{s_{t-W+1}, \dots, s_t\}$ con atención causal (Ec. 2.10), puede construir una representación sensible a la *velocidad y aceleración* del crecimiento de la severidad. Esta interpretación es compatible con el mejor desempeño observado, pero no demuestra causalmente el mecanismo: para ello sería necesario un estudio de ablación que elimine o permute la información histórica.

6.2. Entropía de Shannon como proxy de confianza algorítmica

Un hallazgo conceptualmente relevante de esta tesis no es solo la mejora en $\bar{r}$ sino el patrón de entropía: la H_{norm} del Transformer *baja* justo cuando la amenaza es más grave, y *sube* cuando el entorno se estabiliza. Esto sugiere que la red podría haber aprendido, sin supervisión explícita, un patrón compatible con una forma de *metacognición emergente*; la evidencia no permite afirmar que el agente “sepa cuándo sabe” en sentido humano.

Este comportamiento es cualitativamente compatible con el descrito por Boldt & Yeung [17] y formalizado por Fleming [18] para la confianza humana medida con EEG. La aportación metodológica de la tesis es haber traducido ese biomarcador a una métrica

reproducibile: $1 - H_{\text{norm}}(\pi(a | s))$, calculable en cualquier modelo con salida `softmax`, sin necesidad de hardware de neuroimagen.

6.3. El ensemble como mejor sistema del benchmark

Una lectura ingenua de la Tabla 5.1 podría concluir que la ganancia de `pes_ens` sobre `pes_trf` ($\Delta\bar{r} \approx 0,010$) es marginal. Tres argumentos sugieren lo contrario:

1. **Reducción de varianza.** La desviación estándar cae de 0,045 a 0,035, un $\approx 23\%$ menos. Para una aplicación hipotética, esto se traduciría en un piso peor-caso más alto; el presente trabajo no evalúa despliegues operativos ni riesgo clínico.
2. **Descomposición sesgo-varianza.** El promediado no elimina el sesgo del miembro mediano pero reduce la varianza en un factor $\rho + (1 - \rho)/K$, donde ρ es la correlación entre miembros y $K = 3$. En mPES los miembros son arquitectónicamente distintos (`dqn`, `rdqn`, `trf`), lo que mantiene ρ alejado de uno y materializa la ganancia teórica.
3. **Resiliencia OOD.** En los tres estresores universales (Sec. 6.4), el ensemble obtiene la mejor métrica peor-caso, aportando robustez precisamente en las condiciones más adversas.

En síntesis: el Transformer constituye el mejor punto de partida entre los modelos individuales, mientras que el ensemble obtiene el mejor desempeño agregado dentro del benchmark.

6.4. Los tres estresores universales

Las tres condiciones identificadas en la Sec. 5.9 constituyen el verdadero *núcleo* del benchmark: cualquier discusión de robustez debe priorizarlas frente a las demás celdas. Las columnas estructurales (`struct_*`), por el contrario, exhiben degradación cero y actúan como banda de control que valida la invarianza del propio *esquema* de evaluación antes que la robustez del modelo.

6.5. El falso aprobado de `pes_a2c`

El paquete `pes_a2c` alcanza una performance media competitiva ($\bar{r} \approx 0,887$) pero la divergencia KL de su distribución de acción respecto del agente bajo perturbaciones de severidad es esencialmente cero, según la matriz de divergencia KL del benchmark. En términos prácticos: *el actor ha colapsado a una política casi insensible al contexto* que, por suerte, queda cerca del óptimo en la condición empírica. Este fenómeno — conocido en la literatura como *policy collapse* [12, 13] — ilustra por qué los métodos de gradiente de política sin regularización entrópica fuerte exigen una evaluación *OOD* explícita: una métrica agregada sobre la distribución de entrenamiento es incapaz de detectar el problema.

7. Conclusiones

Esta tesis presentó **mPES**, una plataforma de aprendizaje por refuerzo para la asignación óptima de recursos en una pandemia simulada, y la utilizó como banco de prueba para una comparación rigurosa entre **ocho paquetes implementados**, de los cuales se evaluaron siete modelos: tres tabulares (`pes_base`, `pes_q1`, `pes_dq1`) y cinco profundas (`pes_dqn`, `pes_rdqn`, `pes_a2c`, `pes_trf` y el *ensemble* `pes_ens`).

7.1. Mensajes principales

1. **La arquitectura influye en este escenario, además de los hiperparámetros.**

El salto entre `pes_q1` (Q-Learning optimizado con Optuna) y `pes_trf` (Transformer causal) es estructural: aproximadamente un **35 %** de reducción en severidad residual normalizada, no alcanzable por una mejor búsqueda de hiperparámetros sobre la representación tabular.

2. **Transformer individual, ensemble más robusto.** La auto-atención causal hace de `pes_trf` la mejor arquitectura individual; sus resultados son compatibles con una mejor utilización de la información temporal. El *soft voting* sobre `{dqn, rdqn, trf}` (Ec. 4.7) la supera en el rendimiento agregado: el ensemble reduce la varianza un ≈ 23 % y eleva el piso peor-caso, lo que importa cuando la métrica relevante es el

riesgo y no sólo la media.

3. **La Entropía de Shannon es un proxy interpretable de incertidumbre.** El comportamiento de $H_{\text{norm}}(\pi)$ a lo largo de la trayectoria reproduce cualitativamente los marcadores neurofisiológicos de confianza identificados en el dataset experimental de referencia (ds004477 [17, 18]), validando la analogía cualitativa entre el paradigma neurocognitivo y el algorítmico, no una validación de equivalencia neurofisiológica.
4. **El benchmark es reproducible.** El pipeline `general/scripts/orchestrate.py` regenera todos los CSVs, *heatmaps* y el reporte `benchmark_report.md`, lo que facilita la extensión y la auditoría externa.

7.2. Limitaciones del estudio

Las conclusiones deben interpretarse considerando las limitaciones del diseño experimental. En primer lugar, el benchmark agregado se ejecutó con una *semilla* fija (42) por celda. Aunque las $n = 64$ secuencias por escenario permiten estimar diferencias entre arquitecturas, una réplica con varias semillas sería necesaria para separar con mayor rigor el efecto algorítmico de la variabilidad aleatoria, especialmente en los extremos del ranking.

La limitación principal proviene del propio escenario utilizado como representante de una decisión secuencial compleja. El entorno *Pandemic Scenario* modela de forma controlada una dinámica acumulativa: cada asignación modifica la severidad futura y obliga a administrar un presupuesto limitado a lo largo de una secuencia. En ese sentido, constituye un *testbench* adecuado para estudiar anticipación, memoria y robustez ante cambios de distribución. Sin embargo, sigue siendo una representación simplificada: no reproduce la totalidad de las variables, restricciones y retroalimentaciones presentes en una decisión informada analítica del mundo real.

Por ello, los resultados identifican un ajuste entre arquitectura y estructura del problema, no una superioridad universal de un modelo. Además, la complejidad comportamental de la decisión humana incluye factores como preferencias, interpretación de evidencia, presión social, confianza y deliberación, que no están representados en la política del

agente. La relación entre la entropía algorítmica y la confianza humana debe entenderse, por tanto, como una analogía funcional y como una limitación del propio hallazgo, no como una equivalencia neurofisiológica. La generalización a otros dominios requiere nuevos escenarios y estudios con participación humana.

7.3. Trabajo futuro

- **Réplica multi-semilla.** Repetir el sweep 7×22 con al menos 5 semillas independientes permitiría construir intervalos de confianza alrededor del ranking de la Tabla 5.1 y verificar la estabilidad de los tres estresores universales (Sec. 6.4).
- **Sustituir el escenario simulado** por series temporales reales (p. ej. datos COVID-19 desagregados por región) y evaluar generalización contra los marcos de modelado clásicos [23].
- **Acoplamiento humano-agente.** Diseñar interfaces que comuniquen H_{norm} y la varianza del ensemble al decisor humano y medir su impacto en la calidad de la decisión conjunta (*human-in-the-loop* significativo).
- **Aprender los pesos del ensemble.** Los pesos w_k de la Ecuación (4.7) son hoy proporcionales al desempeño *in-distribution*; una extensión natural es aprender pesos dependientes del contexto $w_k(s)$ mediante una red de *mixture-of-experts* que dirija el voto en función del régimen del brote.

7.4. Cierre

mPES demuestra que, en problemas de decisión bajo crisis con fuerte componente temporal, la elección de la arquitectura es decisiva, y que la combinación de Transformers y Cuantificación de Incertidumbre ofrece a la vez *eficiencia* (mejor política) y *transparencia* (confianza interpretable). Esa doble virtud es lo que distingue un benchmark experimental reproducible de una comparación académica no sistemática.

8. Agradecimientos

A mi director, **Dr. Rodrigo Ramele**, por la guía paciente y rigurosa a lo largo de la Maestría en Ciencia de Datos, esta tesis es deudora directa de su visión sobre el futuro de la tecnología.

A mi esposa, **Analía Giménez**, por su apoyo durante cada etapa de mi vida profesional y cada uno de los proyectos de nuestra vida juntos.

A mis **padres**, que me dieron la oportunidad de formarme como ingeniero y, con ello, los cimientos para hoy acceder a una Maestría. Su esfuerzo silencioso está en el origen de cada uno de mis proyectos.

Referencias Bibliográficas

- [1] BCI-NE Lab, University of Essex. PES: Pandemic experiment scenario. <https://github.com/BCI-NE/PES>, 2022. Financiado por Dstl/UK MoD y US/UK DoD BARI. Accedido: mayo 2026.
- [2] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, Cambridge, MA, 2nd edition, 2018.
- [3] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.
- [4] Hado van Hasselt. Double Q-learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 23, pages 2613–2621, 2010.
- [5] Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *International Conference on Machine Learning (ICML)*, 1999.
- [6] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [7] Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8(3–4):293–321, 1992.
- [8] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- [9] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [10] Matthew Hausknecht and Peter Stone. Deep recurrent Q-learning for partially observable MDPs. In *AAAI Fall Symposium Series*, 2015.
- [11] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992.
- [12] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2016.
- [13] Marcin Andrychowicz, Anton Raichuk, Piotr Stańczyk, Manu Orsini, Sertan Girgin, Raphaël Marinier, Léonard Hussenot, Matthieu Geist, Olivier Pietquin, Marcin Michalski, Sylvain Gelly, and Olivier Bachem. What matters for on-policy deep actor-critic methods? a large-scale study. *International Conference on Learning Representations (ICLR)*, 2021.
- [14] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

- [15] Emilio Parisotto, H. Francis Song, Jack W. Rae, Razvan Pascanu, Caglar Gulcehre, Siddhant M. Jayakumar, Max Jaderberg, Raphael Lopez Kaufman, Aidan Clark, Seb Noury, Matthew M. Botvinick, Nicolas Heess, and Raia Hadsell. Stabilizing transformers for reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2020.
- [16] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [17] Annika Boldt and Nick Yeung. Shared neural markers of decision confidence and error detection. *Journal of Neuroscience*, 35(8):3478–3484, 2015.
- [18] Stephen M. Fleming. Metacognition and confidence: A review and synthesis. *Annual Review of Psychology*, 75:241–268, 2024.
- [19] James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2011.
- [20] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2019.
- [21] Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, Hillsdale, NJ, 2nd edition, 1988.
- [22] Bernard L. Welch. The generalization of Student’s problem when several different population variances are involved. *Biometrika*, 34(1/2):28–35, 1947.
- [23] Ellen Kuhl. *Computational Epidemiology: Data-Driven Modeling of COVID-19*. Springer, 2021.
- [24] BCI-NE Lab. OpenNeuro Dataset ds004477, 2022. URL https://nemar.org/dataexplorer/detail?dataset_id=ds004477. Disponible en NEMAR/OpenNeuro.
- [25] Akashdeep Nijjar, Ittipat Promnorakid, Riccardo Poli, Caterina Cinel, Thomas Reed, Christopher Baker, Stephen Hinton, and Stephen Fairclough. Enhancing decision-making in human-machine teams. In *2025 IEEE International Conference on Metrology for eXtended Reality, Artificial Intelligence and Neural Engineering (MetroXRaine)*, 2025. doi: 10.1109/metroxraine66377.2025.11340006.
- [26] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *AAAI Conference on Artificial Intelligence*, 2018.
- [27] Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2020.
- [28] Charles Packer, Katelyn Gao, Jernej Kos, Philipp Krähenbühl, Vladlen Koltun,

- and Dawn Song. Assessing generalization in deep reinforcement learning. *arXiv:1810.12282*, 2018.
- [29] Robert Kirk, Amy Zhang, Edward Grefenstette, and Tim Rocktäschel. A survey of zero-shot generalisation in deep reinforcement learning. *Journal of Artificial Intelligence Research*, 76:201–264, 2023.
- [30] Maximiliano Vega. mPES: Multiple pandemic experiment suite. <https://github.com/Maximiliano0/mPES>, 2026. Repositorio oficial. Accedido: mayo 2026.
- [31] Mark Towers et al. Gymnasium v1.0: A standard interface for reinforcement learning environments. <https://gymnasium.farama.org/>, 2024.
- [32] Vincy Fon and Francesco Parisi. Litigation and the evolution of legal remedies: A dynamic model. *Public Choice*, 116(3–4):419–433, 2003. doi: 10.1023/A:1024822710849.
- [33] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Apéndice

A. Comandos de reproducción

La Tabla A.1 resume los comandos canónicos que regeneran cada uno de los artefactos reportados en esta tesis. Todos se ejecutan desde la raíz del *workspace* mPES con el entorno virtual activado (*win_mpes_env* en Windows, *linux_mpes_env* en Linux), Python 3.12 y las dependencias fijadas en *utils/config/requirements.txt*.

Cuadro A.1: Comandos para reentrenar los modelos, optimizar hiperparámetros y ejecutar la evaluación del benchmark en cada paquete del repositorio mPES.

Paquete	Entrenamiento	Optimización bayesiana
pes_base	<code>python -m tabular.pes_base</code>	—
pes_ql	<code>python -m tabular.pes_ql</code>	<code>python -m tabular.pes_ql.ext.optimize_rl</code>
pes_dql	<code>python -m tabular.pes_dql</code>	<code>python -m tabular.pes_dql.ext.optimize_rl</code>
pes_dqn	<code>python -m ml.pes_dqn</code>	<code>python -m ml.pes_dqn.ext.optimize_dqn</code>
pes_rdqn	<code>python -m ml.pes_rdqn</code>	<code>python -m ml.pes_rdqn.ext.optimize_rdqn</code>
pes_a2c	<code>python -m ml.pes_a2c</code>	<code>python -m ml.pes_a2c.ext.optimize_a2c</code>
pes_trf	<code>python -m ml.pes_trf</code>	<code>python -m ml.pes_trf.ext.optimize_tr</code>
pes_ens	<code>python -m ml.pes_ens</code>	— (sin parámetros entrenables)

B. Orquestador del benchmark

El barrido completo de $7 \times 22 = 154$ celdas se ejecuta con una sola llamada:

```
python -m general.scripts.orchestrate \
    --models pes_ql pes_dql pes_dqn pes_rdqn pes_a2c pes_trf pes_ens \
    --scenarios all
```

El script:

1. ejecuta cada combinación (*modelo*, *escenario*) reutilizando los pesos entrenados en `<paquete>/outputs/`;
2. agrega los retornos por secuencia con `general/scripts/aggregate.py` para producir las matrices CSV en `general/results/` (`matrix_global_mean.csv`, `matrix_std.csv`, `matrix_cohen_d.csv`, `matrix_welch_p.csv`, `matrix_action_kl.csv`, `matrix_ood_degradation.csv`);

3. dibuja los *heatmaps* con `general/scripts/plot_matrix.py`;
4. emite el reporte ejecutivo `general/results/benchmark_report.md` con `general/scripts/report.py`.

C. Estresores universales

Los tres escenarios identificados en la Sec. 6.4 como estresores universales ($p < 10^{-3}$ en *todos* los modelos) son:

- `sev_extrapolate_high`: severidades por encima del rango *in-distribution*.
- `len_extrapolate_long`: secuencias más largas que las observadas durante entrenamiento.
- `joint_extrap_both`: combinación simultánea de las dos anteriores.

Las tres columnas estructurales (`struct_more_total`, `struct_many_short_blocks`, `struct_few_long_blocks`) exhiben degradación cero y $D_{\text{KL}} = 0$, actuando como banda de control del propio *esquema* de evaluación.